# CS683: Programming Assignment II

24B0925 – G V N S Arya Charan

24B1014 – Jaswith Mavuru

24B1055 – Penugonda Venkata Srinivas

24B1259 – Purna Pritam

---

## Contents

**1 Task 1A: Understanding the IP-Stride Prefetcher** **2**

**2 Task 1B: Stride Pattern Prefetching (SPP)** **7**

2.1 Objective . . . 7

2.2 Design and Implementation . . . 7

2.2.1 Per-IP Access Tracking . . . 7

2.2.2 Uniform and Non-Uniform Pattern Prediction . . . 7

2.2.3 Prefetch Generation and Page Protection . . . 7

2.2.4 Accuracy-based throttling . . . 8

2.3 Storage Overhead . . . 8

2.4 Experimental Methodology . . . 9

2.5 Results . . . 9

2.5.1 IPC and speedup . . . 9

2.5.2 L1D load MPKI, accuracy, and coverage . . . 9

2.6 Discussion . . . 12

2.7 Task 1B Summary . . . 12

**3 Task 2A: Instruction-Conscious L2 Replacement (I-RRIP)** **13**

3.1 Problem Statement and Objective . . . 13

3.2 Background: Re-Reference Interval Prediction . . . 13

3.3 Implementation Overview . . . 13

3.4 Insertion Policy: Why Instruction and Data Lines Differ . . . 14

3.4.1 Why insert instruction lines at RRPV 2? . . . 14

3.4.2 Why insert data lines at RRPV 3? . . . 14

3.4.3 Why use RRPV 3 for writeback insertion? . . . 14

3.5 Promotion Policy: Why Use Different Values on Hits? . . . 15

3.5.1 Why promote instruction hits to RRPV 0? . . . 15

3.5.2 Why promote data hits only to RRPV 2? . . . 15

3.5.3 Why leave the state unchanged on a writeback hit? . . . 15

3.6 Eviction Policy: Why Prefer Data Lines? . . . 16

3.6.1 Step 1: Use an invalid way if available . . . 16

3.6.2 Step 2: Search for a data line at RRPV 3 . . . 16

3.6.3 Step 3: Fall back to any line at RRPV 3 . . . 16

3.6.4 Step 4: Age the RRPVs if no victim is ready . . . 16

3.6.5 Why not always evict data lines? . . . 16

3.6.6 Experimental evidence for data-first eviction . . . 17
3.7 Final Policy Summary . . . 17
3.8 Parameter Exploration . . . 17
3.9 Experimental Results and Selection of Experiment 7 . . . 18
3.9.1 Why Experiment 7 was selected . . . 18
3.9.2 Evidence for the chosen data-hit promotion value . . . 19
3.9.3 Evidence for the chosen instruction insertion value . . . 19
3.9.4 Evidence for data-first victim selection . . . 19
3.10 Experimental Methodology . . . 20
3.11 Baseline Comparison: IPC and Speedup . . . 20
3.12 L2 Load MPKI Comparison . . . 21
3.13 Discussion of the Final Results . . . 22
3.13.1 Reduction in instruction-load MPKI . . . 22
3.13.2 Trade-off with data-load MPKI . . . 22
3.13.3 Effect on combined L2 load MPKI . . . 23
3.13.4 Relationship between MPKI and speedup . . . 23
3.14 Conclusion . . . 23

**4 Task 2B: Dynamic I-RRIP 24**
4.1 Implementation . . . 24
4.1.1 Replacement State and Policy Parameters . . . 24
4.1.2 Set-Dueling and Leader-Set Distribution . . . 24
4.1.3 Access and Miss Tracking . . . 25
4.1.4 Threshold-Based Policy Selection . . . 25
4.1.5 Insertion, Promotion, and Victim Selection . . . 25
4.1.6 Decision Procedure . . . 25
4.2 Experimental Results . . . 26
4.2.1 Results for Trace 1 . . . 26
4.2.2 Results for Trace 2 . . . 26
4.2.3 Results for Trace 3 . . . 27
4.3 IPC Analysis . . . 27
4.4 Speedup Analysis . . . 27
4.5 L2 Cache Load MPKI Analysis . . . 28
4.6 Instruction and Data MPKI Analysis . . . 29
4.6.1 Instruction Load MPKI . . . 29
4.6.2 Data Load MPKI . . . 29
4.7 Rationale for Leader-Set Selection and Decision Logic . . . 30

**5 Task 3: Incorporating SPP with Dynamic I-RRIP 31**
5.1 Objective and Motivation . . . 31
5.2 Implementation and Design Rationale . . . 31
5.2.1 Dynamic Instruction-Conscious Replacement . . . 31
5.2.2 Identifying Prefetched Cache Lines . . . 31
5.2.3 Insertion Policy for Prefetch Packets . . . 31
5.2.4 Promotion Policy for Prefetch Packets . . . 32
5.2.5 Eviction Policy for Unused Prefetches . . . 32

5.3 Experimental Methodology . . . . . 32
5.4 Experimental Results . . . . . 33
5.4.1 IPC and Speedup . . . . . 33
5.4.2 Prefetch Accuracy, Coverage, and Load MPKI . . . . . 33
5.5 Plots . . . . . 34
5.6 Discussion . . . . . 37
5.7 Task 3 Summary . . . . . 37

# 1 Task 1A: Understanding the IP-Stride Prefetcher

## Purpose of the code

The supplied file, `prefetcher/ip_stride.l1d_pref`, implements an instruction-pointer-based stride prefetcher for the L1 data cache (L1D). The basic idea is to learn the memory-access pattern of each instruction pointer (IP). If the same instruction accesses cache lines at a regular distance, the prefetcher predicts later lines and requests them before the processor needs them.

For example, an instruction that accesses cache-line addresses $100, 104, 108, 112, \ldots$ has a constant stride of $+4$ cache lines. After the repeated stride has been observed, the prefetcher can request future lines. This is different from predicting every next line: the prediction distance is based on the stride learned for that IP.

The explanation below follows the source file in the order in which its main components appear. Line numbers refer to the supplied file.

## 1. Constants and the tracker structure (lines 4–30)

```
1  #define IP_TRACKER_COUNT 24
2  #define PREFETCH_DEGREE 3
3  #define MASK_IP 0xFF
4
5  class IP_TRACKER {
6    public:
7      uint64_t ip;
8      uint64_t last_cl_addr;
9      int64_t last_stride;
10     uint32_t lru;
11
12     IP_TRACKER () {
13         ip = 0;
14         last_cl_addr = 0;
15         last_stride = 0;
16         lru = 0;
17     };
18 };
19
20 IP_TRACKER trackers[IP_TRACKER_COUNT];
```

The three constants determine the size of the tracking table, the number of future lines requested for a successful prediction, and how the instruction pointer is represented in the table.

- `IP_TRACKER_COUNT` is 24, so the table contains 24 tracker entries. Each entry remembers the most recent state for one tracked IP.
- `PREFETCH_DEGREE` is 3. When the stride prediction condition is satisfied, the code attempts to prefetch up to three future cache lines. This is a fixed compile-time value.
- `MASK_IP` is `0xFF`. The expression `ip & MASK_IP` keeps only the lowest eight bits of the IP. Consequently, two different IPs with the same lowest eight bits can be treated as the same tracker identity.

Each `IP_TRACKER` entry has four fields. The `ip` field identifies the instruction being tracked. `last_cl_addr` stores the last cache-line address seen for that IP. `last_stride` stores the stride from the most recent non-zero address difference. Finally, `lru` stores recency information used when a table entry must be replaced. The constructor initializes every field to zero; the initial LRU ordering is assigned separately during initialization.

The table is a single global array in this source file. The code does not declare a separate table for each IP: instead, each access searches the array for the matching masked IP.

## 2. Initialisation (lines 33–39)

```
1  void CACHE::l1d_prefetcher_initialize()
2  {
3      cout << "CPU " << cpu
4           << " L1D IP-based stride prefetcher" << endl;
5
6      for (int i = 0; i < IP_TRACKER_COUNT; i++)
```

```
        trackers[i].lru = i;
}
```

ChampSim calls this callback when the L1D prefetcher is initialized. It prints a message identifying the CPU and the prefetcher. It then assigns LRU values from 0 to 23. In this implementation, a smaller LRU value represents a more recently used entry, while the value 23 identifies the entry selected as the least recently used.

The loop only sets the LRU field. The other fields are initialized by the `IP_TRACKER` constructor. At this point, no real access history has been learned.

## 3. Converting the address and looking up an IP (lines 41–52)

```
void CACHE::l1d_prefetcher_operate(
    uint64_t addr, uint64_t ip, uint8_t cache_hit,
    uint8_t type, uint8_t critical_ip_flag)
{
    uint64_t cl_addr = addr >> LOG2_BLOCK_SIZE;
    ip = ip & MASK_IP;

    int index = -1;
    for (index = 0; index < IP_TRACKER_COUNT; index++) {
        if (trackers[index].ip == ip)
            break;
    }
```

The `operate` callback is invoked for memory accesses handled by the cache. Its arguments include the byte address, instruction pointer, and additional access information. In this implementation, the prediction logic uses the address and IP; the other parameters are not used in the shown function body.

The line `addr >> LOG2_BLOCK_SIZE` converts the byte address into a cache-line address. If a cache line contains $2^b$ bytes, shifting right by $b$ removes the byte offset within that line. Strides can then be calculated in cache-line units. The IP is masked before lookup, so the same reduced IP value is used consistently for tracking.

The loop checks each of the 24 entries. If a match is found, `break` leaves the loop with `index` identifying that entry. If no match is found, the loop finishes with `index == IP_TRACKER_COUNT`; the next block handles that case.

## 4. Allocating an entry for a new IP (lines 54–74)

```
if (index == IP_TRACKER_COUNT) {
    for (index = 0; index < IP_TRACKER_COUNT; index++) {
        if (trackers[index].lru == (IP_TRACKER_COUNT - 1))
            break;
    }

    trackers[index].ip = ip;
    trackers[index].last_cl_addr = cl_addr;
    trackers[index].last_stride = 0;

    for (int i = 0; i < IP_TRACKER_COUNT; i++) {
        if (trackers[i].lru < trackers[index].lru)
            trackers[i].lru++;
    }
    trackers[index].lru = 0;

    return;
}
```

When an IP is not found, the prefetcher needs a table entry for it. It searches for the entry whose LRU value is 23, which is the least recently used entry according to the ordering established by this code. That entry is overwritten with the new masked IP and the current cache-line address. Its previous stride is set to zero because there is not yet enough history to establish a stride.

The LRU update then increases the recency values of entries that were more recent than the selected victim, and sets the new entry's LRU value to zero. Thus, the newly allocated entry becomes the most recently used.

The final `return` is important: on the first access for a newly tracked IP, the code does not calculate a stride or issue a prefetch. It only records the starting point. Later accesses from that IP provide the history needed

for prediction.

## 5. Sanity check and signed stride calculation (lines 76–92)

```
if (index == -1)
    assert(0);

int64_t stride = 0;
if (cl_addr > trackers[index].last_cl_addr)
    stride = cl_addr - trackers[index].last_cl_addr;
else {
    stride = trackers[index].last_cl_addr - cl_addr;
    stride *= -1;
}
```

After the new-IP case has returned, the code expects to have found a valid tracker. The assertion is a sanity check for an unexpected state. Under the normal lookup flow, a found entry has an index from 0 through 23.

The stride is the difference between the current cache-line address and the previous one for this IP. The code uses separate branches because the addresses are unsigned values. If the current line is higher, the difference is positive. If it is lower, the difference is calculated in the opposite order and then multiplied by $-1$. This preserves the direction of the access pattern.

For instance, accesses to lines $200, 196, 192$ produce strides of $-4, -4$, not $+4, +4$. The signed stride is later added to the current line address when predicting future accesses.

## 6. Ignoring a zero stride (lines 94–96)

```
if (stride == 0)
    return;
```

A zero stride means that the current access maps to the same cache line as the previous access recorded for this IP. The code returns immediately and does not prefetch. Since no new line position has been observed, this access does not provide a useful non-zero stride for the predictor.

Notice that this early return occurs before the tracker update at the end of the function. Therefore, in this case, the stored address, stored stride, and LRU information are not updated by this invocation.

## 7. Checking whether the stride repeats (lines 98–101)

```
if (stride == trackers[index].last_stride) {
    // prefetching logic follows
}
```

The code only enters the prefetching block when the current stride equals the stride stored from the previous non-zero stride calculation. This is the implementation's simple test for a repeated constant-stride pattern.

Consider accesses to lines $100, 104, 108$. The first access allocates the entry. The next access observes $+4$, but the stored stride was zero, so it does not prefetch. The code then stores $+4$. The following access observes $+4$ again; now the condition succeeds and prefetching begins.

This design is intentionally simple. It does not maintain a history of multiple stride values, nor does it classify recurring non-uniform sequences such as $+1, +2, +1, +2, \ldots$. That limitation is relevant to the later SPP task.

## 8. Generating future addresses (lines 102–110)

```
for (int i = 0; i < PREFETCH_DEGREE; i++) {
    uint64_t pf_address =
        (cl_addr + (stride * (i + 1))) << LOG2_BLOCK_SIZE;

    if ((pf_address >> LOG2_PAGE_SIZE) !=
        (addr >> LOG2_PAGE_SIZE))
        break;

    // the prefetch request is issued below
}
```

The loop attempts to generate three addresses because the degree is three. For each iteration, the current cache-line address is advanced by one, two, or three multiples of the learned stride. The expression is shifted left by `LOG2_BLOCK_SIZE` to convert the predicted cache-line address back into a byte address suitable for the prefetch API.

The page check compares the predicted address's page number with the current demand access's page number. In this code, the page size is represented by `LOG2_PAGE_SIZE`, which is normally 12 for a 4 KB page. If a predicted address is outside the current page, the loop breaks. Since the predictions move progressively along the same signed stride, the code stops issuing further predictions for this access once the first boundary crossing is detected.

This restriction prevents this prefetcher from intentionally generating a request across the current 4 KB page boundary.

## 9. Choosing L1 or L2 as the fill destination (lines 111–116)

```
if (MSHR.occupancy < (MSHR.SIZE >> 1))
    prefetch_line(ip, addr, pf_address, FILL_L1, 0);
else
    prefetch_line(ip, addr, pf_address, FILL_L2, 0);
```

The Miss Status Holding Register (MSHR) tracks outstanding memory requests. The code compares its current occupancy with half of its capacity. When occupancy is below half, the predicted line is requested with `FILL_L1`, allowing it to be placed in L1. Otherwise, the request uses `FILL_L2`, avoiding an L1 fill when MSHR resources are more heavily occupied.

This is a simple pressure-based choice: under lower MSHR occupancy, the prefetcher is more willing to place data in L1; under higher occupancy, it directs the request to L2 instead. The code does not change the prefetch degree based on this condition; it changes the requested fill level.

## 10. Updating the tracker and LRU state (lines 119–127)

```
trackers[index].last_cl_addr = cl_addr;
trackers[index].last_stride = stride;

for (int i = 0; i < IP_TRACKER_COUNT; i++) {
    if (trackers[i].lru < trackers[index].lru)
        trackers[i].lru++;
}
trackers[index].lru = 0;
return;
```

After the stride and prefetching checks, the current cache-line address and the newly calculated non-zero stride are saved. The LRU state is then updated to mark this tracker as the most recently used entry. Entries that were more recent than this tracker have their LRU values increased, and the current entry is set to zero.

This update lets the table adapt as different IPs access memory. If all 24 entries are in use and a new IP arrives, the entry that has gone the longest without being used is selected for replacement.

## 11. Cache-fill callback and final statistics (lines 130–139)

```
void CACHE::l1d_prefetcher_cache_fill(
    uint64_t v_addr, uint64_t addr, uint32_t set,
    uint32_t way, uint8_t prefetch, uint64_t v_evicted_addr,
    uint64_t evicted_addr, uint32_t metadata_in)
{
    return;
}

void CACHE::l1d_prefetcher_final_stats()
{
    cout << "CPU " << cpu
         << " L1D PC-based stride prefetcher final stats" << endl;
    cout << "Degree: " << PREFETCH_DEGREE << endl;
```

```
}
```

The cache-fill callback returns immediately and does not update any prefetcher-specific state. In particular, the implementation does not record whether a prefetch was later used. The final-statistics callback prints a label and the configured degree; it does not calculate an accuracy value itself.

## 12. Worked example

Suppose one IP accesses the following cache-line addresses:

$$100, \quad 104, \quad 108, \quad 112, \quad 116.$$

The first access allocates a tracker with the last address set to 100 and the stored stride set to zero. The access to 104 observes $+4$, but does not trigger prefetching because $+4 \neq 0$. The access to 108 observes $+4$ again, which matches the stored stride. At this point, the code tries to prefetch lines 112, 116, and 120 relative to the current line 108, subject to the page-boundary condition. The same logic continues for later accesses while the stride remains constant.

For a decreasing sequence such as $300, 296, 292, 288, \ldots$, the stride is $-4$. Once this value repeats, predicted addresses move backwards from the current cache line. The sign of the stride therefore matters for correct prediction.

## 13. Summary of the implementation

| **Component** | **What the code does** |
|---|---|
| Tracking table | Stores 24 IP entries with the last line, last stride, and LRU rank. |
| IP lookup | Masks the IP to eight bits and linearly searches the table. |
| Learning | Calculates signed differences between successive cache-line addresses. |
| Prediction condition | Issues prefetches when the current non-zero stride matches the previously stored stride. |
| Prefetch degree | Attempts up to three future lines. |
| Page-boundary check | Stops if a predicted address leaves the current 4 KB page. |
| Fill destination | Uses L1 when MSHR occupancy is below half; otherwise uses L2. |
| Feedback | The fill callback does not track usefulness or accuracy. |

Table 1: Summary of the supplied IP-stride implementation.

The main strength of this prefetcher is its low-complexity approach to regular memory access patterns. It tracks a small amount of history per IP and can predict multiple lines after observing a repeated stride. Its limitations are the small tracker table, the eight-bit IP mask, the fixed prefetch degree, and its inability to learn repeating non-uniform stride sequences. Also, because the cache-fill callback does not track usefulness, this source file does not implement accuracy-based throttling.

# 2 Task 1B: Stride Pattern Prefetching (SPP)

## 2.1 Objective

We have to implement an L1 data-cache prefetcher that can learn memory-access patterns associated with individual instruction pointers (IPs). A conventional IP-stride prefetcher works well when an instruction repeatedly accesses addresses separated by a constant stride. It is less suitable when the stride changes in a repeating sequence. SPP addresses this limitation by supporting both uniform and non-uniform stride patterns.

The implementation is required to keep prefetch requests within the current OS page, issue more than one prefetch when appropriate, and adapt its degree according to observed prefetch accuracy. The evaluation compares SPP with both a no-prefetching baseline and the supplied IP-stride prefetcher.

## 2.2 Design and Implementation

### 2.2.1 Per-IP Access Tracking

SPP maintains a table of 64 entries per CPU to track memory accesses from different instruction pointers (IPs). Each entry stores the masked IP, last cache-line address, previous stride, pattern signature, confidence information, prediction mode, and LRU rank.

When a new IP is encountered, the prefetcher uses an invalid entry if one is available. Otherwise, it replaces the lru entry. The current cache-line address is recorded, and the prefetcher waits for further accesses before making predictions.

Only the lowest 16 bits of the IP are stored. This reduces storage overhead, but different IPs with identical lower 16 bits may share a tracker. The implementation supports stride magnitudes of at most 63 cache lines. If a stride exceeds this limit, the history for that IP is reset and learning starts again.

### 2.2.2 Uniform and Non-Uniform Pattern Prediction

SPP supports two types of access patterns:

- **Uniform pattern:** When the same stride is observed repeatedly, the uniform-confidence value increases. Once the required confidence is reached, the prefetcher predicts future cache lines using the learned stride, provided non-uniform mode is not active.
- **Non-uniform pattern:** When the stride changes, SPP updates a signature using the previous signature and the newly observed stride. This signature indexes a pattern table containing a predicted next stride and its confidence.

A matching pattern-table entry increases its confidence, while a mismatch reduces it. An entry whose confidence reaches zero can be updated with the newly observed stride. SPP follows a non-uniform prediction only when the corresponding entry has a confidence of at least two.

The signature is calculated using a shift, XOR, and a 10-bit mask. It provides a compact representation of stride history rather than storing the complete sequence. This saves storage, although different stride histories can map to the same signature.

### 2.2.3 Prefetch Generation and Page Protection

The initial prefetch degree is three. At each prediction step, SPP either continues with the learned uniform stride or follows the predicted non-uniform stride sequence. Prefetch generation stops when no valid next stride is available, the predicted stride is zero, or the predicted address crosses the current page boundary.

The implementation also checks MSHR occupancy before generating prefetches. If occupancy reaches `MSHR.SIZE - 2`, prefetch generation is skipped for that access. Otherwise, the fill destination is selected as follows:

- **L1 fill:** Used when MSHR occupancy is below half of its capacity.
- **L2 fill:** Used when MSHR occupancy is at least half of its capacity.

This mechanism limits additional requests when MSHR resources are nearly full and directs prefetches to L2 when occupancy is higher.

#### 2.2.4 Accuracy-based throttling

The prefetcher updates its degree after an epoch of 1,024 L1D accesses. It uses the change in prefetch fills and useful prefetches over the epoch to estimate accuracy. A degree update is made when at least eight fills have been observed in that epoch; otherwise, the previous degree is retained, except for the initial no-fill case, which resets the degree to three.

The implementation uses the following thresholds and degree choices:

Table 2: Accuracy-based prefetch-degree selection in the supplied SPP implementation.

| Observed accuracy | Selected prefetch degree |
|---|---|
| Accuracy $\geq 70\%$ | 4 |
| $40\% \leq$ accuracy $< 70\%$ | 3 |
| $20\% \leq$ accuracy $< 40\%$ | 2 |
| Accuracy $< 20\%$ | 1 |

These thresholds are heuristic operating points in the current implementation, not values proven optimal before the experiments. The intended trade-off is to allow more aggressive prefetching when a high fraction of prefetched lines are useful, and to reduce the number of predicted lines when accuracy is low. The results below will show whether this choice improves performance for the evaluated traces.

### 2.3 Storage Overhead

The following calculation counts the logical bits represented by the declared fields. It excludes compiler-inserted structure padding and alignment. The tables and bookkeeping state are allocated per CPU.

Table 3: Bit-level storage of one IP-tracker entry.

| Field | Field width (bits) | Count |
|---|---|---|
| Valid flag | 8 | 1 |
| Masked IP | 16 | 1 |
| Last cache-line address | 64 | 1 |
| Last stride | 8 | 1 |
| Stride signature | 16 | 1 |
| Uniform confidence | 8 | 1 |
| Non-uniform-mode flag | 8 | 1 |
| LRU rank | 8 | 1 |
| Total per tracker | 136 | 1 |
| Number of tracker entries | 136 | 64 |

Table 4: Bit-level storage of one pattern-table entry.

| Field | Field width (bits) | Count |
|---|---|---|
| Predicted stride | 8 | 1 |
| Confidence counter | 8 | 1 |
| Total per pattern entry | 16 | 1 |
| Number of pattern entries | 16 | 1,024 |

The resulting logical table storage per CPU is:

$$S_{\text{tracker}} = 64 \times 136 = 8{,}704 \text{ bits},$$
$$S_{\text{pattern}} = 1{,}024 \times 16 = 16{,}384 \text{ bits},$$
$$S_{\text{tables}} = 8{,}704 + 16{,}384 = 25{,}088 \text{ bits}.$$

This is 3,136 bytes, or approximately 3.06 KiB, of logical table storage per CPU. The per-CPU bookkeeping variables add 576 logical bits: a 32-bit epoch access counter, three 64-bit fill/useful counters, a 32-bit degree variable, and five 64-bit degree counters. Including this bookkeeping gives 25,664 bits (3,208 bytes, approximately 3.13 KiB) per CPU, before compiler padding and alignment. The table-only figure is the primary prefetch-table overhead.

## 2.4 Experimental Methodology

All configurations use the same ChampSim build configuration and the same trace inputs. For every trace, run 25 million warm-up instructions and then 25 million instructions in the region of interest, as specified in the assignment. The three configurations are:

- **Baseline:** data prefetching disabled.
- **IP-stride:** the supplied `ip_stride.l1d_pref`.
- **SPP:** the implementation described in this section.

For each run, record IPC, L1D load MPKI, prefetch accuracy, and the information needed to calculate L1D load-miss coverage. Use the same definition of prefetch accuracy across the prefetcher runs. The speedup is relative to the no-prefetching baseline as

$$\text{Speedup} = \frac{\text{IPC}_{\text{prefetcher}}}{\text{IPC}_{\text{baseline}}}.$$

L1D load MPKI should be taken from the ChampSim statistics for the region of interest. Coverage should be calculated using the assignment's definition:

$$\text{Coverage} = \frac{\text{L1D load misses}_{\text{baseline}} - \text{L1D load misses}_{\text{prefetcher}}}{\text{L1D load misses}_{\text{baseline}}} \times 100\%.$$

Use the actual L1D load-miss counts for this calculation, rather than mixing counts from different regions of interest. If the baseline miss count is zero, coverage is undefined and should be reported as N/A.

## 2.5 Results

### 2.5.1 IPC and speedup

Table 5: IPC and speedup for the three configurations.

| Trace | Metric | Baseline | IP-stride | SPP |
|---|---|---|---|---|
| Trace 1 | IPC | 0.232113 | 0.233108 | 0.233993 |
| | Speedup | 1.000 | 1.004 | 1.008 |
| Trace 2 | IPC | 0.249982 | 0.250141 | 0.250874 |
| | Speedup | 1.000 | 1.001 | 1.004 |
| Trace 3 | IPC | 0.381868 | 0.382687 | 0.387866 |
| | Speedup | 1.000 | 1.002 | 1.016 |

### 2.5.2 L1D load MPKI, accuracy, and coverage

Accuracy and coverage are not meaningful for the no-prefetching baseline; marked those cells as N/A. For the baseline, recorded the L1D load MPKI and the L1D load-miss count which are used in the coverage calculation.

Table 6: L1D load MPKI and prefetch metrics.

| Trace | Metric | Baseline | IP-stride | SPP |
|---|---|---|---|---|
| Trace 1 | L1D load MPKI | 10.6788 | 10.1193 | 9.67792 |
| | L1D load misses | 266,969 | 252,983 | 241,948 |
| | Prefetch accuracy (%) | N/A | 52.9313 | 53.8499 |
| | L1D coverage (%) | N/A | 5.2388 | 9.3722 |
| Trace 2 | L1D load MPKI | 13.5372 | 13.4384 | 13.2654 |
| | L1D load misses | 338,431 | 335,959 | 331,635 |
| | Prefetch accuracy (%) | N/A | 37.9151 | 44.0735 |
| | L1D coverage (%) | N/A | 0.7304 | 2.0081 |
| Trace 3 | L1D load MPKI | 3.40356 | 3.31768 | 2.88736 |
| | L1D load misses | 85,089 | 82,942 | 72,184 |
| | Prefetch accuracy (%) | N/A | 16.0642 | 44.6404 |
| | L1D coverage (%) | N/A | 2.5232 | 15.1665 |

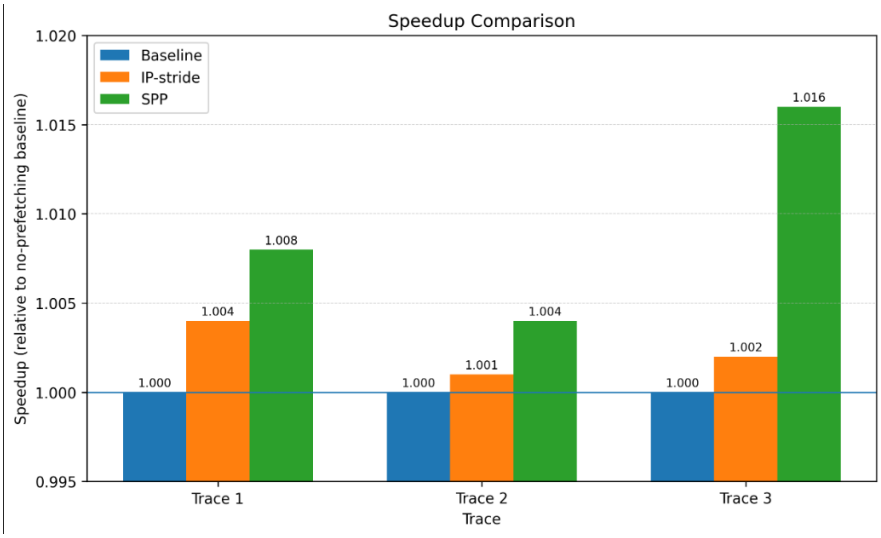



Figure 1: Speedup Comparision

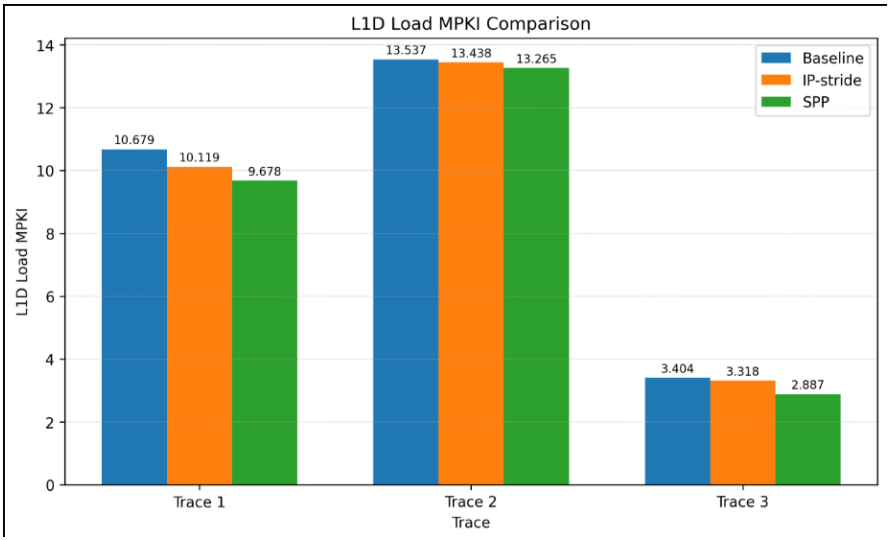



Figure 2: L1D Load MPKI Comparision

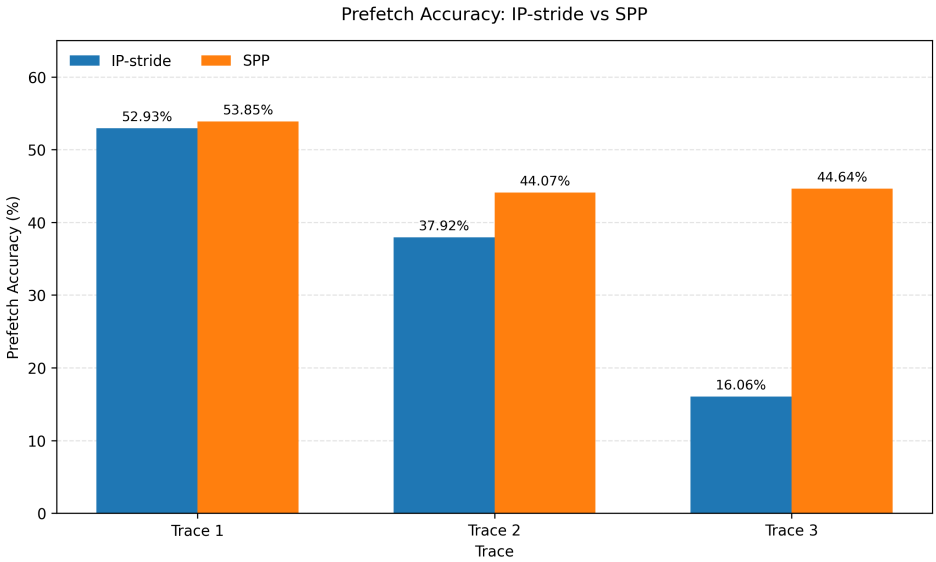



Figure 3: Prefetch Accuracy: IP-Stride vs SPP

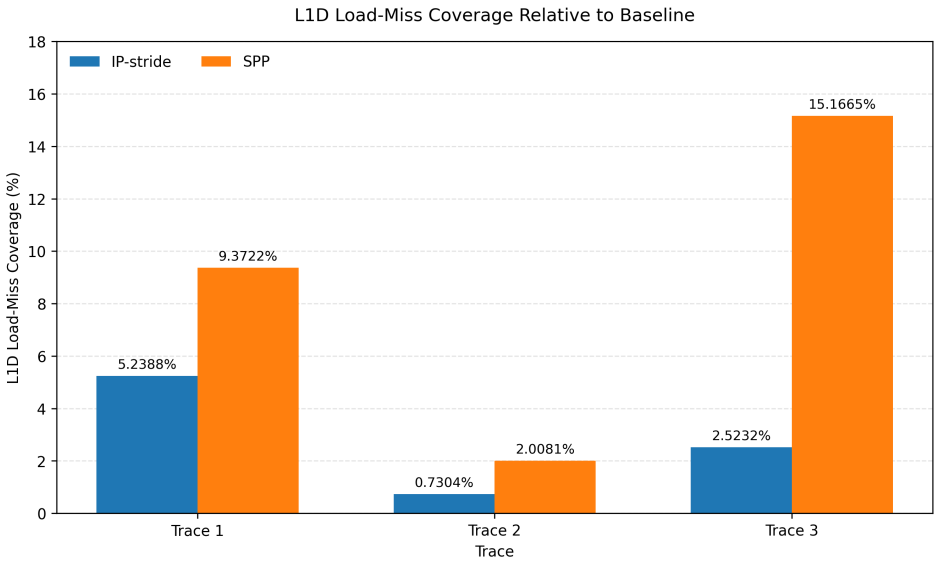



Figure 4: L1D Load-Miss Coverage

## 2.6 Discussion

Across all three traces, SPP outperforms the no-prefetching baseline in both IPC and L1D load MPKI, and it also performs better than the supplied IP-stride prefetcher. The magnitude of the improvement varies across traces.

For Trace 1, SPP achieves an IPC of 0.233993, compared with 0.233108 for IP-stride and 0.232113 for the baseline. This corresponds to a speedup of approximately 1.008× over the baseline, compared with 1.004× for IP-stride. SPP also reduces L1D load MPKI from 10.6788 for the baseline and 10.1193 for IP-stride to 9.67792. Its prefetch accuracy is 53.8499%, slightly higher than the 52.9313% achieved by IP-stride. Its load-miss coverage is also higher, at 9.3722% compared with 5.2388%. These results indicate that SPP's higher coverage and slightly better accuracy are consistent with its greater reduction in load misses and its modest IPC improvement.

For Trace 2, the improvements are smaller. SPP achieves an IPC of 0.250874, compared with 0.250141 for IP-stride and 0.249982 for the baseline. The corresponding speedups are approximately 1.004× for SPP and 1.001× for IP-stride. L1D load MPKI decreases from 13.5372 for the baseline to 13.4384 with IP-stride and 13.2654 with SPP. SPP records 44.0735% prefetch accuracy and 2.0081% load-miss coverage, compared with 37.9151% accuracy and 0.7304% coverage for IP-stride. Although SPP improves both metrics, its coverage remains relatively low, and the resulting performance gain is correspondingly modest.

Trace 3 shows the largest benefit from SPP. Its IPC is 0.387866, compared with 0.382687 for IP-stride and 0.381868 for the baseline, giving a speedup of approximately 1.016× over the baseline. SPP reduces L1D load MPKI from 3.40356 for the baseline and 3.31768 for IP-stride to 2.88736. Its accuracy reaches 44.6404%, substantially higher than IP-stride's 16.0642%, while its coverage increases from 2.5232% to 15.1665%. The substantial coverage improvement is consistent with the larger reduction in load misses and the stronger IPC improvement observed for this trace.

Overall, SPP improves IPC and reduces L1D load MPKI for every trace. Its accuracy and coverage are higher than those of IP-stride in all three cases, although the relationship between these metrics and IPC is not necessarily proportional because overall performance depends on factors beyond L1D load misses alone. The measurements support the effectiveness of SPP for these traces, particularly Trace 3, but they do not establish that the selected thresholds or prefetch degrees are optimal.

The accuracy-based throttling policy is designed to regulate prefetch aggressiveness: high measured accuracy permits a degree of four, whereas low accuracy reduces the degree to one. The measured accuracies are 53.8499%, 44.0735%, and 44.6404% for SPP on Traces 1, 2, and 3, respectively. These values indicate that the high-accuracy threshold of 70% is not reached by the reported overall accuracies. However, overall accuracy alone is insufficient to determine how the degree changed during execution, because the policy may adapt across individual measurement epochs. The reported degree counters should therefore be used to verify whether the degree changed as intended. If the counters show transitions to lower degrees during low-accuracy epochs and higher degrees during more accurate epochs, this would be consistent with the design of the throttling policy. The measurements presented here do not, by themselves, demonstrate that the chosen thresholds are optimal.

## 2.7 Task 1B Summary

The SPP implementation combines per-IP tracking, uniform-stride prediction, signature-based non-uniform prediction, page-boundary protection, MSHR-aware prefetching, and accuracy-based degree control. Across all three evaluated traces, SPP achieves higher IPC and lower L1D load MPKI than both the no-prefetching baseline and the supplied IP-stride prefetcher.

SPP achieves speedups of approximately 1.008×, 1.004×, and 1.016× over the baseline on Traces 1, 2, and 3, respectively. It reduces L1D load MPKI from 10.6788 to 9.67792 on Trace 1, from 13.5372 to 13.2654 on Trace 2, and from 3.40356 to 2.88736 on Trace 3. SPP also achieves higher prefetch accuracy and L1D load-miss coverage than IP-stride on every trace, with the most substantial coverage improvement occurring on Trace 3.

These results demonstrate that SPP provides consistent performance improvements over the evaluated baseline and IP-stride configurations. The gains are modest on Traces 1 and 2 but more pronounced on Trace 3. The implementation also incorporates mechanisms intended to control prefetch aggressiveness and limit resource pressure. Its storage overhead is approximately 3.13 KiB according to the implementation's storage accounting. Overall, the results support the effectiveness of SPP for the evaluated workloads, although they do not establish that its throttling thresholds are optimal.

# 3 Task 2A: Instruction-Conscious L2 Replacement (I-RRIP)

## 3.1 Problem Statement and Objective

The objective of Task 2A is to implement an instruction-conscious replacement policy for the L2 cache. Instruction and data cache lines compete for the same cache capacity, even though their access patterns and impact on processor performance can differ.

Instruction lines contain the instructions that the processor frontend must fetch and decode. If an instruction line is evicted and accessed again, the resulting cache miss can delay instruction delivery to the execution pipeline. Data lines are also important, but the objective of this task is specifically to reduce the number of L2 instruction-load misses by giving instruction lines a higher retention priority.

A conventional replacement policy does not necessarily distinguish between instruction and data lines when deciding which line to retain. Consequently, an instruction line may be evicted even when a data line could have been selected instead. The proposed Instruction-Conscious Re-Reference Interval Prediction (I-RRIP) policy addresses this issue by incorporating instruction-type information into the insertion, promotion, and eviction decisions.

The primary evaluation metric is L2 instruction-load MPKI, which measures the number of L2 instruction-load misses per thousand instructions. The objective is to minimise this value. Speedup relative to LRU is also measured to determine whether the reduction in instruction-load misses translates into an improvement in overall processor performance.

## 3.2 Background: Re-Reference Interval Prediction

RRIP uses a Re-Reference Prediction Value (RRPV) to estimate how soon a cache line is likely to be accessed again. In this implementation, the RRPV is represented using two bits and ranges from 0 to 3.

| RRPV | Interpretation |
|---|---|
| 0 | Strongest retention priority; predicted to be reused soon. |
| 1 | High retention priority. |
| 2 | Lower retention priority. |
| 3 | Maximum predicted re-reference distance; preferred eviction candidate. |

Table 7: Interpretation of the RRPV values used by I-RRIP.

When a cache set requires a victim, RRIP looks for a line whose RRPV equals the maximum value, 3. If no line has this value, the policy ages the existing lines by increasing their RRPVs, up to 3, and repeats the search. This ageing mechanism eventually produces an eligible victim without requiring a separate stack of recency information.

The key modification in I-RRIP is that instruction and data lines do not necessarily receive the same RRPV. Their initial values and their hit-promotion values are chosen according to the objective of protecting instruction lines while still allowing the cache to retain useful data.

## 3.3 Implementation Overview

The policy is implemented in `replacement/irrip.l2c_repl`. The replacement-state update function is extended with an additional `uint8_t is_instr` argument, which indicates whether the incoming request is an instruction access.

The policy uses this information during insertion and hit updates. For eviction, it examines the instruction-type information associated with the lines already present in the selected cache set. This distinction is necessary because the type of the incoming request and the types of the resident lines serve different purposes.

The policy makes decisions at three stages:

1. **Insertion:** Assign an initial RRPV when a line enters the cache.
2. **Promotion:** Update the RRPV when a resident line is accessed again.
3. **Eviction:** Select a victim when a cache set requires space for a new line.

These decisions work together. Insertion establishes the initial retention priority, promotion incorporates evidence of reuse, and eviction determines which line loses its place when cache capacity is insufficient.

## 3.4 Insertion Policy: Why Instruction and Data Lines Differ

Insertion is important because a newly inserted cache line has not yet demonstrated that it will be reused. Giving every new line a very low RRPV would protect it strongly, even if it were accessed only once. On the other hand, inserting every line at the maximum RRPV would provide no initial protection to instruction lines that are likely to be accessed again.

The final I-RRIP configuration therefore uses different insertion values for instruction and data lines.

| **Line type** | **Insertion RRPV** | **Reasoning** |
|---|---|---|
| Instruction line | 2 | Gives instruction lines an initial advantage over data lines without placing every newly fetched instruction at the strongest retention level. |
| Data line | 3 | Treats a newly inserted data line as a possible eviction candidate until further accesses provide evidence of reuse. |
| Writeback line | 3 | Uses the maximum RRPV for writeback insertion, consistent with the policy's instruction-retention objective. |

Table 8: Insertion decisions in the final I-RRIP configuration.

### 3.4.1 Why insert instruction lines at RRPV 2?

An instruction line is inserted at RRPV 2 rather than at the maximum RRPV of 3. This gives it a better initial retention priority than a new data line.

However, the policy does not insert instruction lines at RRPV 0. Doing so would give every newly inserted instruction line the strongest possible protection before the policy has observed whether the line will be reused. A one-time instruction access could then occupy cache space that might be needed by a frequently reused line.

RRPV 2 provides a compromise. A newly inserted instruction line receives some protection, but it can still become a victim under sufficient cache pressure. If the line is accessed again, the promotion policy can move it to RRPV 0.

The experimental results also support this choice in the final policy configuration. Experiment 6 used RRPV 1 for instruction insertion, whereas Experiment 7 used RRPV 2, with the other listed parameters unchanged. Experiment 7 reduced instruction-load MPKI on all three traces. Thus, in this configuration, giving a new instruction line a moderate initial advantage and reserving the strongest protection for a line that has demonstrated reuse produced better results.

### 3.4.2 Why insert data lines at RRPV 3?

Data lines are inserted at RRPV 3, the maximum RRPV. This makes a new data line eligible for eviction if the set is under pressure and the line has not subsequently demonstrated reuse.

This decision is deliberate rather than an assertion that data lines are unimportant. The policy is designed to prioritise instruction-line retention, and the shared L2 capacity means that giving data lines the same initial protection could weaken that objective.

A data line that is accessed again is promoted to RRPV 2. Therefore, the policy does not permanently penalise a data line: it allows subsequent accesses to improve its retention priority, but gives it less protection than an instruction line.

### 3.4.3 Why use RRPV 3 for writeback insertion?

Writeback lines are also inserted at RRPV 3. This keeps the insertion policy consistent with the objective of prioritising instruction lines and avoids assigning additional retention priority to a writeback line merely because it was inserted.

This is a design choice evaluated within the implemented policy; it does not imply that every writeback line is useless or that a different writeback policy could never perform better.

## 3.5 Promotion Policy: Why Use Different Values on Hits?

A hit provides evidence that a resident line has been reused. The replacement policy should incorporate this information rather than treating every line according only to its insertion state.

However, I-RRIP does not promote instruction and data lines equally. Instruction hits receive the strongest promotion, while data hits receive a more moderate promotion.

| Event | New RRPV | Reasoning |
|---|---|---|
| Instruction hit | 0 | A repeated instruction access provides evidence of reuse, so the line receives the strongest available retention priority. |
| Data hit | 2 | A data hit is still rewarded, but the line remains less protected than an instruction line after a hit. |
| Writeback hit | Unchanged | The replacement state is not promoted merely because a writeback hit occurs. |

Table 9: Promotion decisions in the final I-RRIP configuration.

### 3.5.1 Why promote instruction hits to RRPV 0?

The objective is not to protect every instruction line equally at all times. Instead, the policy aims to retain instruction lines that show evidence of reuse.

When an instruction line is hit, its RRPV is set to 0. This is the strongest retention state in the implementation. It distinguishes a line that has already been reused from a newly inserted line whose future usefulness is uncertain.

This decision helps the policy allocate retention priority according to observed behaviour. An instruction line that is repeatedly accessed can remain protected, while an instruction line that is never reused can still be evicted if the set eventually requires space.

The decision also works with the instruction insertion policy. Insertion at RRPV 2 gives a new instruction line a moderate advantage, and a later hit moves it to RRPV 0. This two-stage approach avoids giving the strongest protection to every new instruction line while allowing reused instructions to obtain it.

### 3.5.2 Why promote data hits only to RRPV 2?

Data hits are promoted to RRPV 2 rather than to RRPV 0. A data hit should improve a line's position relative to a newly inserted data line, but I-RRIP must also account for the competing objective of retaining instruction lines.

If data hits were promoted as aggressively as instruction hits, frequently accessed data lines could occupy more cache capacity at the expense of instruction lines. This would weaken the intended instruction-conscious behaviour of the policy.

RRPV 2 offers a compromise. It rewards a data line for being accessed again, but keeps it closer to the maximum RRPV than an instruction line that has been hit. Consequently, data lines remain eligible for eviction sooner under the same general cache pressure.

The parameter experiments support this decision. Experiments 5 and 6 differ in their data-hit promotion value: Experiment 5 promotes data hits to RRPV 1, while Experiment 6 promotes them to RRPV 2. Experiment 6 has lower instruction-load MPKI on all three traces.

The same pattern is observed when comparing Experiments 7 and 8. Experiment 7 uses data-hit RRPV 2, while Experiment 8 uses RRPV 1. Experiment 7 again has lower instruction-load MPKI on all three traces.

These two comparisons support using RRPV 2 for data-hit promotion in the final configuration. The choice is based on the measured instruction-load MPKI objective, rather than on the assumption that data hits should never be rewarded.

### 3.5.3 Why leave the state unchanged on a writeback hit?

The implementation leaves the RRPV unchanged on a writeback hit. A writeback-related hit is not treated as an ordinary demand access that must promote the line to a more protected state.

This avoids changing retention priority for an event that the policy does not use as evidence of the same kind of demand reuse as an instruction or data hit. It also keeps the replacement-state transition simple and deterministic.

## 3.6 Eviction Policy: Why Prefer Data Lines?

Insertion and promotion affect a line's RRPV, but a separate decision is required when the cache set is full. The final I-RRIP policy combines RRIP's maximum-RRPV victim selection with an instruction-conscious preference for evicting data lines.

### 3.6.1 Step 1: Use an invalid way if available

If the set contains an invalid way, the policy uses it instead of evicting a valid cache line. This avoids unnecessary replacement when free capacity is available.

### 3.6.2 Step 2: Search for a data line at RRPV 3

When all ways are valid, the policy first searches for a data line whose RRPV equals 3.

RRPV 3 is already the state used by RRIP to identify lines with the largest predicted re-reference distance. If both instruction and data lines have reached this state, preferring a data line allows the policy to protect an instruction line without selecting a line with a better RRPV than the ordinary RRIP victim candidate.

The data-first rule therefore adds instruction-type awareness to the victim-selection decision. Among the lines that have reached the maximum RRPV, the policy attempts to evict data before instruction.

### 3.6.3 Step 3: Fall back to any line at RRPV 3

If no data line has RRPV 3, the policy selects any line whose RRPV equals 3. This fallback is necessary because instruction lines must remain replaceable.

A policy that absolutely forbids instruction eviction could fail to make progress when a set contains only instruction lines, or when instruction lines are the only candidates that have reached the maximum RRPV. I-RRIP avoids this problem by preferring data eviction without permanently pinning instruction lines in the cache.

### 3.6.4 Step 4: Age the RRPVs if no victim is ready

If no line has RRPV 3, the policy increases the RRPVs below the maximum value and repeats victim selection. This preserves the basic ageing procedure of RRIP.

The ageing step is important because the policy must eventually select a victim even when the current RRPVs are all below 3. Without it, the replacement operation could fail to identify a candidate.

The complete victim-selection procedure is therefore:

1. Select an invalid way if one exists.
2. Otherwise, select a data line at RRPV 3, if available.
3. If no data line is eligible, select any line at RRPV 3.
4. If no line has RRPV 3, age the RRPVs and repeat the search.

### 3.6.5 Why not always evict data lines?

The objective is to prioritise instruction retention, not to make instruction lines irreplaceable. A data-first rule applies only when the policy is selecting among eligible maximum-RRPV candidates. If there is no eligible data line, an instruction line can still be evicted.

This distinction is important because a cache policy must continue to make progress even when its preferred victim class is unavailable. The fallback retains the normal RRIP replacement mechanism while adding a preference for data eviction when possible.

### 3.6.6 Experimental evidence for data-first eviction

Experiment 1 and Experiment 4 have the same insertion and hit-promotion values in the parameter table. The main change is that Experiment 4 enables data-first victim selection.

The instruction-load MPKI decreases from 10.18140 to 9.95760 on Trace 1, from 16.13470 to 12.76560 on Trace 2, and from 1.09420 to 1.02872 on Trace 3.

This comparison shows that enabling data-first victim selection improved the target metric on all three tested traces in that configuration. It provides experimental justification for including the rule in the final design.

The result does not establish that data-first eviction must improve every workload. Its effect depends on the instruction/data mix and the replacement states of the resident cache lines. However, the measured reduction across all three traces supports the decision for this assignment.

## 3.7 Final Policy Summary

The final I-RRIP policy uses the following combination of decisions.

| Operation | Final decision | Purpose |
|---|---|---|
| Initialisation | RRPV 3 | Start from the maximum re-reference distance. |
| Instruction insertion | RRPV 2 | Give instruction lines moderate initial protection. |
| Data insertion | RRPV 3 | Leave data lines more exposed to replacement until reuse is observed. |
| Writeback insertion | RRPV 3 | Keep writeback insertion at the maximum RRPV. |
| Instruction hit | RRPV 0 | Strongly retain instruction lines after a demand hit. |
| Data hit | RRPV 2 | Reward data reuse without matching instruction-hit protection. |
| Writeback hit | Unchanged | Avoid promoting the line for a writeback hit. |
| Victim selection | Data-first at RRPV 3 | Prefer data eviction among maximum-RRPV candidates. |

Table 10: Summary of the selected I-RRIP design.

The rationale behind the policy is that the initial insertion state should not determine a line's retention priority forever. Instruction lines receive a moderate advantage at insertion, strong protection after a hit, and preference during victim selection. Data lines can still be promoted when reused, but they start at the maximum RRPV and receive less aggressive promotion. These decisions give the policy several opportunities to favour instruction retention rather than relying on only one mechanism.

## 3.8 Parameter Exploration

Eight experiments were conducted to evaluate different combinations of instruction insertion, data insertion, hit promotion, and victim-selection rules. All experiments used a maximum RRPV of 3, with writeback hits leaving the replacement state unchanged.

| Experiment | Instr. fill | Data fill | WB fill | Instr. hit | Data hit | Data-first eviction |
|---|---|---|---|---|---|---|
| 1 | 2 | 2 | 2 | 0 | 0 | No |
| 2 | 1 | 2 | 2 | 0 | 0 | No |
| 3 | 2 | 2 | 2 | 0 | 2 | No |
| 4 | 2 | 2 | 2 | 0 | 0 | Yes |
| 5 | 1 | 3 | 3 | 0 | 1 | Yes |
| 6 | 1 | 3 | 3 | 0 | 2 | Yes |
| 7 | 2 | 3 | 3 | 0 | 2 | Yes |
| 8 | 2 | 3 | 3 | 0 | 1 | Yes |

Table 11: Parameter configurations evaluated for I-RRIP.

Experiments 1–4 examined the effects of instruction insertion, data-hit promotion, and data-first victim selection while using the same initial RRPV for data and writeback fills. Experiments 5–8 used RRPV 3 for data and writeback insertion, enabling a stronger difference between the initial treatment of instruction and data lines.

The experiments were evaluated using the same three traces. The primary comparison criterion was the L2 instruction-load MPKI. Speedup relative to LRU was also recorded to measure the overall performance impact of each configuration.

## 3.9 Experimental Results and Selection of Experiment 7

The speedup for each trace is calculated as

$$\text{Speedup}_i = \frac{\text{IPC}_{\text{experiment},i}}{\text{IPC}_{\text{LRU},i}},$$

where $i$ represents Trace 1, Trace 2, or Trace 3. The geometric mean combines the per-trace speedups, normalising each trace against its own LRU baseline.

The following table reports only speedup and L2 instruction-load MPKI for the eight tested experiments. The instruction-load MPKI values are the primary selection criterion.

| | **Speedup relative to LRU** | | | | **L2 instruction-load MPKI** | | |
|---|---|---|---|---|---|---|---|
| **Experiment** | **Trace 1** | **Trace 2** | **Trace 3** | **Geometric mean** | **Trace 1** | **Trace 2** | **Trace 3** |
| 1 | 1.0011× | 1.0122× | 1.0001× | 1.0045× | 10.18140 | 16.13470 | 1.09420 |
| 2 | 1.0001× | 1.0187× | 1.0003× | 1.0063× | 9.97844 | 14.87550 | 1.01880 |
| 3 | 1.0008× | 1.0182× | 1.0000× | 1.0063× | 9.97692 | 14.31860 | 1.04344 |
| 4 | 1.0011× | 1.0260× | 1.0002× | 1.0090× | 9.95760 | 12.76560 | 1.02872 |
| 5 | 0.9971× | 1.0375× | 1.0052× | 1.0131× | 9.22112 | 9.68996 | 0.74888 |
| 6 | 0.9973× | 1.0390× | 1.0061× | 1.0140× | 9.11116 | 9.42000 | 0.71420 |
| **7 (final)** | **1.0062×** | **1.0505×** | **1.0074×** | **1.0212×** | **8.95360** | **8.39448** | **0.68732** |
| 8 | 1.0054× | 1.0476× | 1.0066× | 1.0197× | 9.12124 | 8.85380 | 0.72996 |

Table 12: Speedup relative to LRU and L2 instruction-load MPKI for the eight I-RRIP experiments. Lower instruction-load MPKI is preferred.

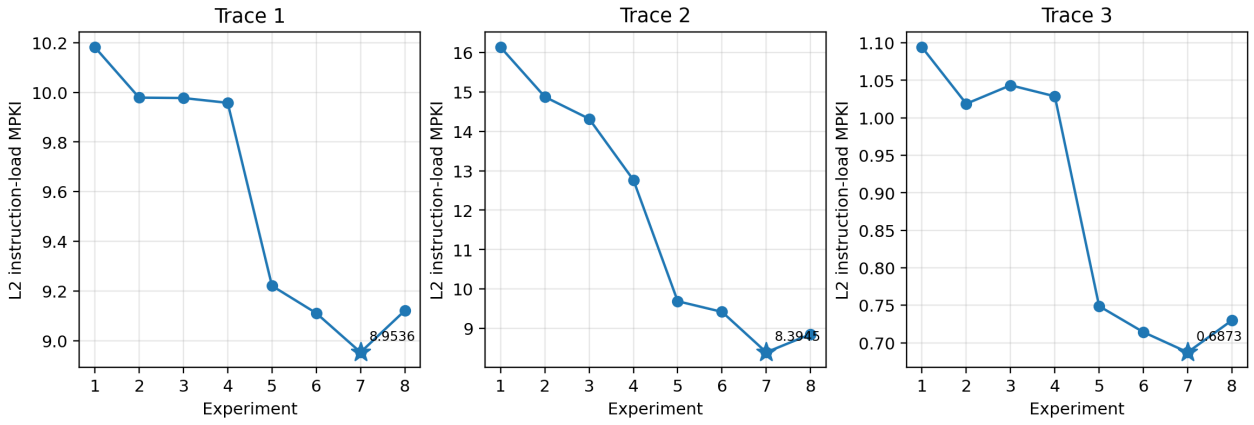



Figure 5: Instruction-Load MPKI Across Experiments (lower is better)

The experiment-sweep plot makes the selection criterion explicit. Because Trace 3 has a much smaller instruction-load MPKI scale than Traces 1 and 2, the traces are shown in separate panels so that the differences between experiments remain visible for every workload.

### 3.9.1 Why Experiment 7 was selected

Experiment 7 was selected primarily because it achieved the lowest L2 instruction-load MPKI among all eight experiments on each of the three traces. Its values were 8.95360 for Trace 1, 8.39448 for Trace 2, and 0.68732 for Trace 3.

The selection process prioritises the requested objective rather than choosing a configuration solely because it achieves the highest IPC. Speedup is used as a secondary metric to check whether the reduction in instruction-load misses also benefits overall performance.

Experiment 7 also achieved a geometric-mean speedup of 1.0212× relative to LRU. Its per-trace speedups were 1.0062×, 1.0505×, and 1.0074×. Thus, in these experiments, the configuration chosen for its low instruction-load MPKI also improved IPC on all three traces.

### 3.9.2 Evidence for the chosen data-hit promotion value

Experiments 5 and 6 differ in their data-hit RRPV: Experiment 5 uses 1, while Experiment 6 uses 2. Experiment 6 produces lower instruction-load MPKI on all three traces.

Experiments 7 and 8 make the same comparison under the final insertion configuration. Experiment 7 uses RRPV 2 for data hits, and Experiment 8 uses RRPV 1. The instruction-load MPKI is again lower with RRPV 2 on each trace.

This repeated result supports using RRPV 2 for data-hit promotion in the final policy. It indicates that, for the tested configurations, more aggressive data promotion did not help achieve the instruction-load MPKI objective.

### 3.9.3 Evidence for the chosen instruction insertion value

Experiments 6 and 7 differ only in instruction insertion: Experiment 6 uses RRPV 1, while Experiment 7 uses RRPV 2. Experiment 7 achieves lower instruction-load MPKI on every trace.

This shows that the strongest initial protection is not necessarily the best insertion choice. A moderate initial advantage for instruction lines, combined with strong promotion after an instruction hit, performed better in the final configuration.

The effect of an insertion value can depend on the other replacement decisions. For example, Experiments 1 and 2 also show a different outcome for instruction insertion under a different set of parameters. Therefore, the selection is based on the complete policy configuration and its measured results, rather than assuming one insertion value is always optimal in every possible policy.

### 3.9.4 Evidence for data-first victim selection

Experiments 1 and 4 have the same listed insertion and hit-promotion values, but Experiment 4 enables data-first victim selection. Its instruction-load MPKI is lower on all three traces.

This comparison supports the decision to choose a data line first when a data line and an instruction line are both eligible at the maximum RRPV. The rule helps preserve instruction lines without preventing their eviction when no eligible data victim exists.

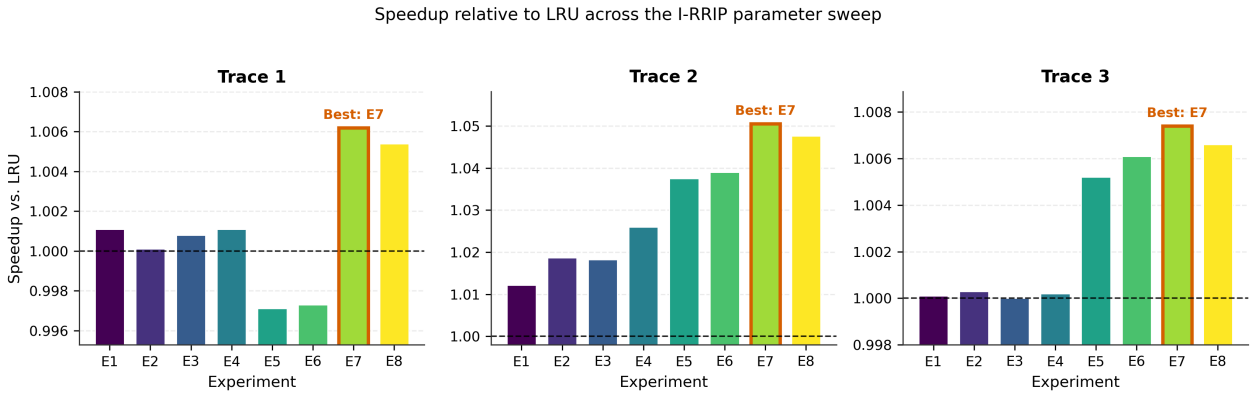



Figure 6: Speedup relative to LRU across the eight I-RRIP experiments.

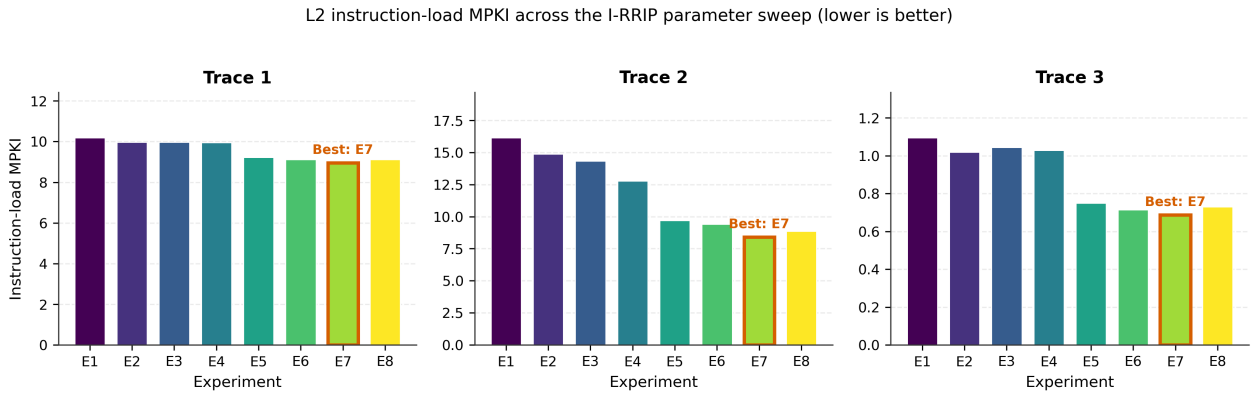



Figure 7: L2 instruction-load MPKI across the I-RRIP parameter sweep (lower is better)

## 3.10 Experimental Methodology

The experiments use ChampSim with 25 million warm-up instructions followed by 25 million instructions in the region of interest on one CPU. Results were collected for `Trace1.gz`, `Trace2.gz`, and `Trace3.gz`.

The final I-RRIP configuration was compared against the LRU and SRRIP baselines. IPC and MPKI values were taken from the region-of- interest statistics in the corresponding simulation logs.

MPKI denotes misses per thousand instructions. L2C load MPKI is the combined L2 load-miss metric, whereas L2 instruction-load MPKI and L2 data-load MPKI describe the instruction and data components, respectively. To meet the Task 2A deliverables, this report presents IPC, speedup, L2C load MPKI, L2 instruction-load MPKI, and L2 data-load MPKI in tables and plots. The policy-comparison figures use one panel per trace because the metric scales differ substantially across the three workloads.

## 3.11 Baseline Comparison: IPC and Speedup

| Trace | LRU IPC | SRRIP IPC | I-RRIP IPC | SRRIP speedup | I-RRIP speedup |
|---|---|---|---|---|---|
| Trace 1 | 0.232264 | 0.232199 | 0.233705 | 0.9997× | 1.0062× |
| Trace 2 | 0.250806 | 0.251481 | 0.263478 | 1.0027× | 1.0505× |
| Trace 3 | 0.381942 | 0.381596 | 0.384758 | 0.9991× | 1.0074× |

Table 13: IPC and speedup relative to LRU for the final I-RRIP configuration.

I-RRIP improves IPC over LRU on all three traces. The largest improvement is observed on Trace 2, where speedup reaches 1.0505×. The other two traces show smaller but positive improvements.

These results provide a performance check for the configuration selected using instruction-load MPKI. They do not replace the primary selection criterion: the reason for choosing Experiment 7 remains its lower instruction-load MPKI across all three traces.

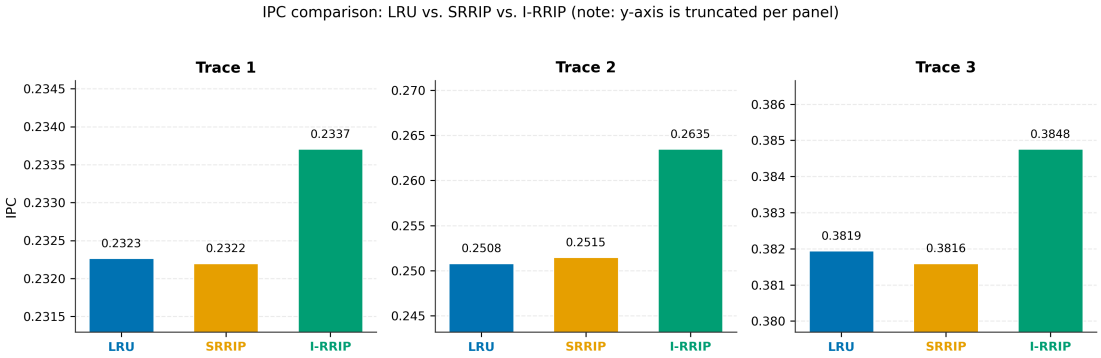



Figure 8: IPC comparison for LRU, SRRIP, and the final I-RRIP configuration.

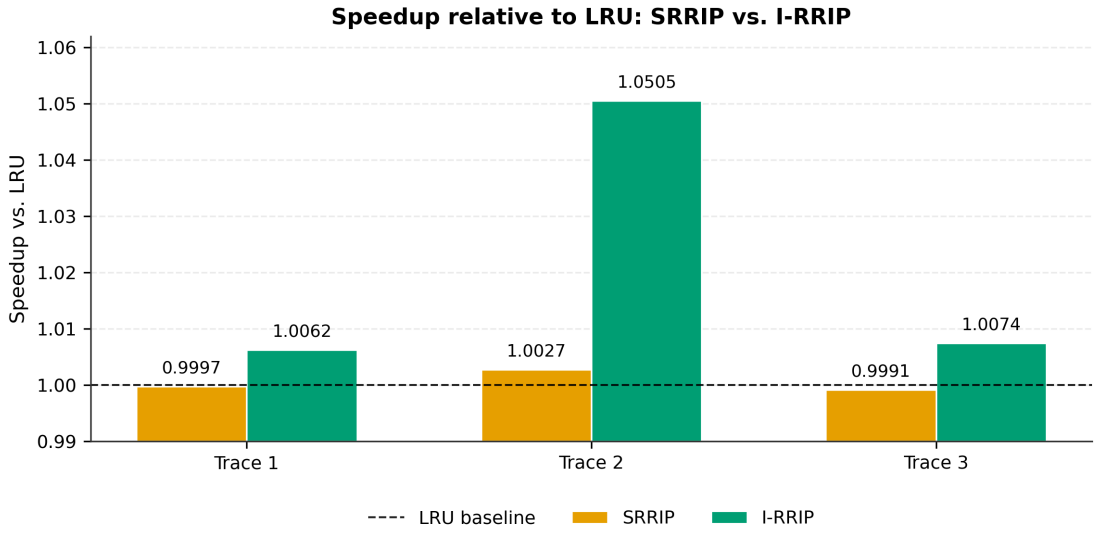



Figure 9: Speedup relative to LRU for the final I-RRIP configuration.

## 3.12 L2 Load MPKI Comparison

| Trace | Policy | L2C load MPKI | L2 instruction-load MPKI | L2 data-load MPKI |
|---|---|---|---|---|
| Trace 1 | LRU | 17.6539 | 10.3374 | 7.3165 |
| Trace 1 | SRRIP | 17.6008 | 10.5392 | 7.0616 |
| Trace 1 | I-RRIP | 18.6511 | 8.9536 | 9.6975 |
| Trace 2 | LRU | 28.1662 | 19.5031 | 8.6631 |
| Trace 2 | SRRIP | 26.4646 | 18.3948 | 8.0698 |
| Trace 2 | I-RRIP | 19.6097 | 8.3945 | 11.2152 |
| Trace 3 | LRU | 3.9558 | 1.1252 | 2.8306 |
| Trace 3 | SRRIP | 4.0442 | 1.1996 | 2.8446 |
| Trace 3 | I-RRIP | 3.8751 | 0.6873 | 3.1878 |

Table 14: L2 load MPKI comparison for LRU, SRRIP, and I-RRIP.

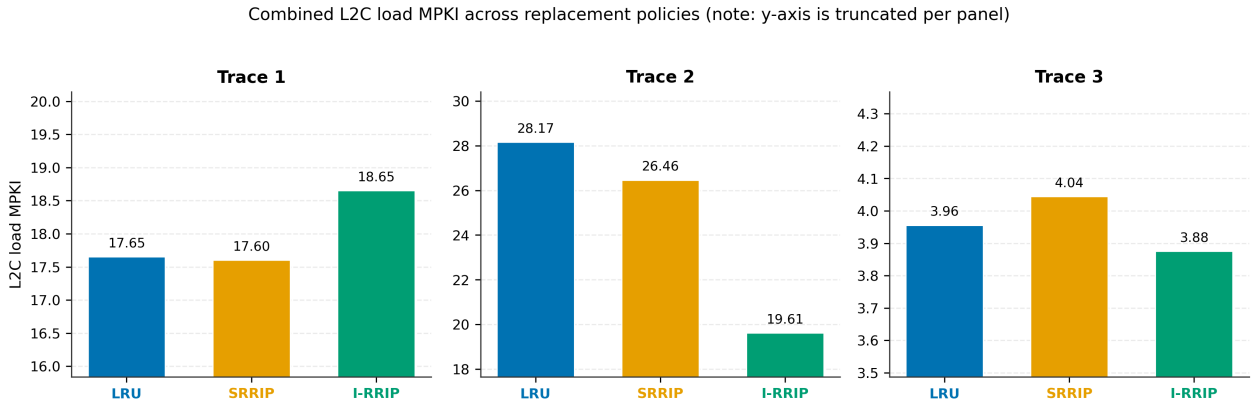



Figure 10: Combined L2C load MPKI for the three replacement policies.

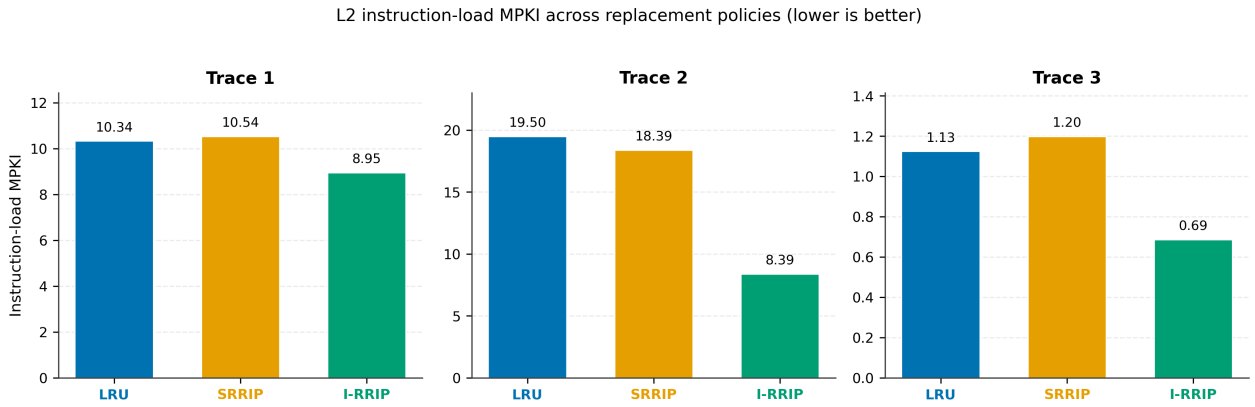



Figure 11: L2 instruction-load MPKI for the three replacement policies.

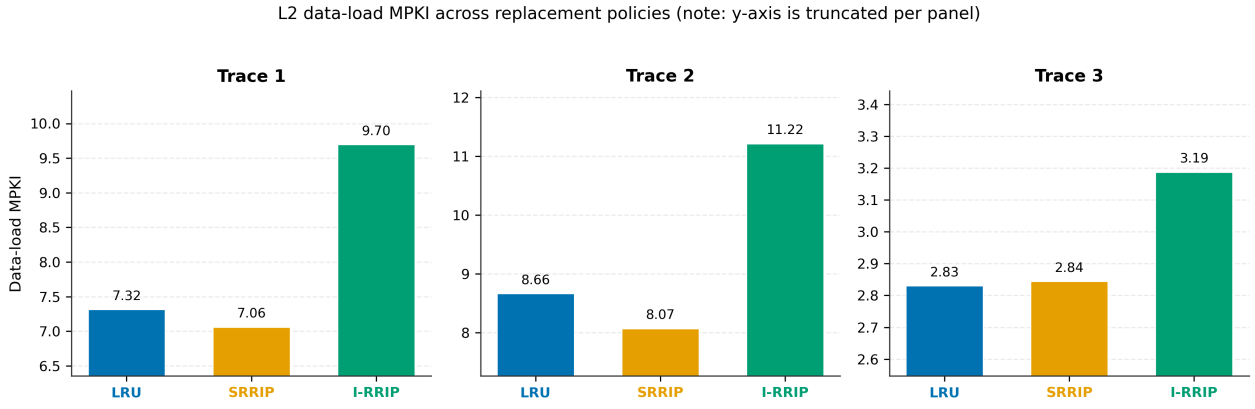



Figure 12: L2 data-load MPKI for the three replacement policies.

## 3.13 Discussion of the Final Results

### 3.13.1 Reduction in instruction-load MPKI

Compared with LRU, the final I-RRIP configuration reduces L2 instruction-load MPKI on every trace.

On Trace 1, instruction-load MPKI decreases from 10.3374 to 8.9536, a reduction of approximately 13.39%. On Trace 2, it decreases from 19.5031 to 8.3945, a reduction of approximately 56.96%. On Trace 3, it decreases from 1.1252 to 0.6873, a reduction of approximately 38.91%.

These improvements are consistent with the design objective. Instruction lines receive moderate initial protection, strong promotion after an instruction hit, and preference during victim selection when a data line is also eligible for eviction.

### 3.13.2 Trade-off with data-load MPKI

Instruction and data lines share the available L2 capacity. Prioritising instruction lines can therefore increase the replacement pressure on data lines.

The measured data-load MPKI increases from 7.3165 to 9.6975 on Trace 1, from 8.6631 to 11.2152 on Trace 2, and from 2.8306 to 3.1878 on Trace 3.

This trade-off is important to report. I-RRIP does not improve every cache-miss metric. Instead, it explicitly prioritises instruction-line retention, accepting that data lines may be evicted more frequently.

#### 3.13.3 Effect on combined L2 load MPKI

The combined L2C load MPKI increases on Trace 1, from 17.6539 to 18.6511. It decreases on Trace 2, from 28.1662 to 19.6097, and on Trace 3, from 3.9558 to 3.8751.

The different outcomes show that the combined effect of replacement depends on each trace's instruction and data access behaviour. The large reduction in instruction-load MPKI on Trace 2 is accompanied by a substantial decrease in its combined L2 load MPKI. On Trace 1, the increase in data-load MPKI outweighs the instruction-load MPKI reduction in the combined metric.

Therefore, the correct conclusion is that I-RRIP meets its main objective of reducing instruction-load MPKI on all three traces, while its effect on total L2 load misses depends on the workload.

#### 3.13.4 Relationship between MPKI and speedup

MPKI and IPC measure different aspects of processor behaviour. MPKI measures the frequency of cache misses per thousand instructions, while IPC measures the overall instruction throughput.

The final I-RRIP configuration improves IPC on all three traces even though combined L2 load MPKI increases on Trace 1. This demonstrates why the replacement policy should be evaluated using both the specified instruction-load MPKI objective and the overall performance metric.

The results do not imply that lowering instruction-load MPKI will always improve IPC for every workload. They show that, in the three evaluated traces, the final configuration reduces instruction-load MPKI and also provides positive speedup relative to LRU.

### 3.14 Conclusion

Task 2A implements I-RRIP by adding instruction-type awareness to the insertion, promotion, and eviction stages of RRIP. Instruction lines are inserted at RRPV 2 and promoted to RRPV 0 on a hit. Data and writeback lines are inserted at RRPV 3, while data hits promote lines to RRPV 2. Writeback hits leave the replacement state unchanged. When eviction is required, data lines at the maximum RRPV are preferred, with a fallback to instruction lines when no eligible data line exists.

These choices reflect a deliberate balance. Newly inserted instruction lines receive moderate protection; instruction hits receive the strongest protection; data lines can still gain protection through reuse; and data-first victim selection favours instruction retention without permanently pinning instruction lines in the cache.

Eight configurations were evaluated. Experiment 7 was selected primarily because it achieved the lowest L2 instruction-load MPKI on all three traces. Its geometric-mean speedup relative to LRU was 1.0212×. Although data-load MPKI increased, the selected configuration achieved the intended instruction-load MPKI reduction and improved IPC on all three evaluated traces.

# 4 Task 2B: Dynamic I-RRIP

Instruction-conscious replacement can reduce instruction-fetch misses, but aggressively retaining instruction lines may increase data misses. Dynamic I-RRIP addresses this trade-off by dynamically selecting between instruction-conscious I-RRIP and conventional RRIP based on a metric measured using a fixed group of leader sets. Leader sets always use baseline RRIP, while follower sets switch to I-RRIP only when the measured metric exceeds a configurable threshold.

All experiments use 25 million warmup instructions followed by 25 million simulation instructions, as required by the assignment. The threshold sensitivity study evaluates 0.10, 0.20, 0.25, 0.30, and 0.40. The implemented configuration uses a threshold of 0.20.

## 4.1 Implementation

### 4.1.1 Replacement State and Policy Parameters

Each L2 cache way maintains a two-bit Re-Reference Prediction Value (RRPV), represented by `l2_rrpv[set][way]`. The maximum RRPV is 3, with larger values indicating that a cache line is predicted to be reused less soon. When a set has no invalid way, the replacement policy selects a line with RRPV = 3. If no such line exists, all RRPVs below the maximum are incremented until a victim becomes available.

The implemented policy uses the parameters listed in Table 15.

Table 15: Dynamic I-RRIP configuration parameters

| Parameter | Implemented value |
|---|---|
| Maximum RRPV | 3 |
| RRIP insertion RRPV | 2 |
| RRIP hit RRPV | 0 |
| I-RRIP instruction insertion RRPV | 2 |
| I-RRIP data insertion RRPV | 3 |
| I-RRIP writeback insertion RRPV | 3 |
| I-RRIP instruction hit RRPV | 0 |
| I-RRIP data hit RRPV | 2 |
| Leader-set stride | 32 |
| Decision threshold | 0.20 |
| Counter epoch limit | 8192 total accesses |
| Data-first victim tie-break in I-RRIP | Enabled |
| Writeback hit leaves RRPV unchanged | Enabled |

### 4.1.2 Set-Dueling and Leader-Set Distribution

The helper function `is_leader_set(set)` identifies leader sets using the condition

$$\text{leader}(s) \iff (s \bmod 32) = 0.$$

Thus, sets $0, 32, 64, \ldots$ are designated as leader sets. For a cache containing 1024 sets, this selects 32 leader sets, corresponding to 3.125% of all sets. The remaining 992 sets, or 96.875%, are follower sets. Table 16 summarizes this distribution.

Table 16: Set distribution in Dynamic I-RRIP for a 1024-set cache

| Set category | Number of sets | Fraction |
|---|---|---|
| Leader sets | 32 | 3.125% |
| Follower sets | 992 | 96.875% |
| Total | 1024 | 100% |

Leader sets always use conventional RRIP and provide observations about the workload's instruction and data access behavior. The resulting metric determines the replacement policy used by follower sets. This design

keeps the leader-set behavior fixed while allowing the majority of cache sets to adapt to the observed access pattern. A stride of 32 also distributes the leader sets regularly across the indexed set space.

#### 4.1.3 Access and Miss Tracking

Four global counters are maintained: `leader_instr_accesses`, `leader_instr_misses`, `leader_total_accesses`, and `leader_total_misses`. These counters are updated only for accesses to leader sets that are not writebacks.

For every eligible leader-set access, the total-access counter is incremented. On a miss, the total-miss counter is incremented as well. If the access is an instruction access, the instruction-access counter is incremented; if that instruction access misses, the instruction-miss counter is also incremented. Therefore, instruction events contribute to both the instruction-specific counters and the corresponding total counters. Writebacks are excluded from the training data.

Let $I_A$ and $I_M$ represent the instruction access and miss counters, respectively, and let $T_A$ and $T_M$ represent the total access and miss counters. The implementation computes the decision metric as

$$R = \frac{I_A + I_M}{T_A + T_M}.$$

If the denominator is zero, the function `use_irrip_for_follower()` returns false, so follower sets initially use baseline RRIP. Otherwise, the computed ratio is compared with the configured threshold.

When `leader_total_accesses` reaches `EPOCH_LIMIT = 8192`, all four counters are right-shifted by one, effectively halving their values. This reduces the influence of older observations and allows more recent workload behavior to influence the decision. Since the limit is checked after each eligible leader-set access, this mechanism performs counter aging rather than resetting counters at fixed intervals.

#### 4.1.4 Threshold-Based Policy Selection

For a follower set, the replacement mode is determined by

$$\text{mode}(s) = \begin{cases} \text{I-RRIP}, & R > \theta, \\ \text{RRIP}, & R \leq \theta, \end{cases}$$

where $\theta$ is the configured threshold. The comparison is strictly greater-than, meaning that a ratio equal to the threshold selects conventional RRIP. Leader sets always remain in RRIP mode.

The policy decision is evaluated during both victim selection and replacement-state updates. Consequently, follower-set victim selection, insertion, and promotion behavior follow the currently selected replacement mode.

#### 4.1.5 Insertion, Promotion, and Victim Selection

In baseline RRIP mode, a non-writeback hit resets the line's RRPV to 0, while a newly filled line is inserted with RRPV 2. A writeback hit leaves the RRPV unchanged.

In I-RRIP mode, instruction fills use RRPV 2, whereas data fills use RRPV 3. Instruction hits reset the RRPV to 0, while data hits reset it to 2. Writeback fills use RRPV 3, and writeback hits leave the RRPV unchanged when the corresponding option is enabled.

During I-RRIP victim selection, the implementation first searches for a non-instruction line with the maximum RRPV. This gives data lines priority for eviction when such a candidate exists. If no suitable candidate is available, the normal maximum-RRPV victim search is used. Baseline RRIP uses only the normal victim search. Under both policies, invalid ways are selected before searching for a victim.

These choices aim to protect instruction lines from premature eviction while allowing data lines with less favorable reuse predictions to be evicted first. However, this prioritization may increase data misses, making the dynamic policy-selection mechanism important for balancing the two types of accesses.

#### 4.1.6 Decision Procedure

The implementation follows these steps:

1. Initialize every RRPV to 3 and reset all leader-set counters.

2. On each non-writeback access to a leader set, update the total access and miss counters and, when applicable, the instruction access and miss counters.
3. When the total-access counter reaches 8192, halve all four counters.
4. Keep leader sets in baseline RRIP mode. For follower sets, compute $R$ and select I-RRIP if $R > \theta$; otherwise, select RRIP.
5. Apply the selected policy's insertion, hit-promotion, and victim-selection rules.

## 4.2 Experimental Results

The evaluation reports Instructions Per Cycle (IPC), speedup, total L2 load MPKI, instruction load MPKI, and data load MPKI for three traces. Speedup is calculated relative to the baseline IPC for the corresponding trace:

$$\text{Speedup} = \frac{\text{IPC of Dynamic I-RRIP}}{\text{IPC of baseline}}.$$

The baseline row represents the reference configuration and is not a threshold setting. All experiments use 25 million warmup instructions and 25 million simulation instructions.

### 4.2.1 Results for Trace 1

Table 17: Dynamic I-RRIP results for Trace1.gz

| Threshold | IPC | Speedup (×) | L2C Load MPKI | Instruction MPKI | Data MPKI |
|---|---|---|---|---|---|
| Baseline | 0.232199 | 1.0000 | 17.6008 | 10.5392 | 7.06164 |
| 0.10 | 0.233599 | 1.006029 | 18.5887 | 9.0060 | 9.58272 |
| 0.20 | 0.233760 | 1.006723 | 18.5689 | 8.9889 | 9.58004 |
| 0.25 | 0.233697 | 1.006451 | 18.5884 | 9.0020 | 9.58632 |
| 0.30 | 0.233736 | 1.006619 | 18.5888 | 9.0034 | 9.58540 |
| 0.40 | 0.232884 | 1.002950 | 18.0610 | 9.3330 | 8.72808 |

For Trace1, the highest IPC is 0.233760 at threshold 0.20, corresponding to a speedup of approximately 0.67%. Instruction MPKI decreases from 10.5392 in the baseline to 8.9889, while data MPKI increases from 7.06164 to 9.58004. Total L2 load MPKI also increases. Thus, the instruction-miss reduction does not translate into a reduction in total misses, although a modest IPC improvement is observed.

### 4.2.2 Results for Trace 2

Table 18: Dynamic I-RRIP results for Trace2.gz

| Threshold | IPC | Speedup (×) | L2C Load MPKI | Instruction MPKI | Data MPKI |
|---|---|---|---|---|---|
| Baseline | 0.251481 | 1.0000 | 26.4646 | 18.3948 | 8.06980 |
| 0.10 | 0.263192 | 1.046568 | 19.7140 | 8.6654 | 11.04860 |
| 0.20 | 0.263195 | 1.046580 | 19.5767 | 8.5286 | 11.04810 |
| 0.25 | 0.262080 | 1.042146 | 19.6680 | 8.7459 | 10.92210 |
| 0.30 | 0.261499 | 1.039836 | 19.8709 | 9.0355 | 10.83540 |
| 0.40 | 0.260946 | 1.037637 | 20.3047 | 9.5955 | 10.70920 |

Trace2 benefits the most from Dynamic I-RRIP. The highest IPC is 0.263195 at threshold 0.20, representing approximately 4.66% speedup over baseline. Total L2 load MPKI falls from 26.4646 to 19.5767, and instruction MPKI decreases substantially from 18.3948 to 8.5286. Data MPKI increases from 8.06980 to 11.04810, indicating that the improvement in instruction behavior comes with a data-miss penalty. Nevertheless, the reduction in total misses and the increase in IPC indicate that the trade-off is beneficial for this workload.

#### 4.2.3 Results for Trace 3

Table 19: Dynamic I-RRIP results for Trace3.gz

| Threshold | IPC | Speedup (×) | L2C Load MPKI | Instruction MPKI | Data MPKI |
|---|---|---|---|---|---|
| Baseline | 0.381596 | 1.0000 | 4.0442 | 1.1996 | 2.84460 |
| 0.10 | 0.384632 | 1.007956 | 3.8683 | 0.7005 | 3.16780 |
| 0.20 | 0.384607 | 1.007891 | 3.8691 | 0.7045 | 3.16460 |
| 0.25 | 0.384613 | 1.007906 | 3.8696 | 0.7040 | 3.16560 |
| 0.30 | 0.382534 | 1.002458 | 3.9334 | 0.8697 | 3.06372 |
| 0.40 | 0.382229 | 1.001659 | 3.9152 | 0.9446 | 2.97060 |

For Trace3, the highest IPC is 0.384632 at threshold 0.10, corresponding to approximately 0.80% speedup. Thresholds 0.20 and 0.25 produce nearly identical IPC values. At threshold 0.20, total L2 load MPKI decreases from 4.0442 to 3.8691, and instruction MPKI decreases from 1.1996 to 0.7045. Data MPKI increases from 2.84460 to 3.16460. The overall performance improvement is positive but relatively small compared with Trace2.

### 4.3 IPC Analysis

Figure 13 compares IPC across the five threshold settings for all three traces. For Trace1, IPC is highest at threshold 0.20, with a value of 0.233760. Trace2 also achieves its highest reported IPC at 0.20, with a value of 0.263195, effectively tied with 0.10 at 0.263192. Trace3 performs marginally best at 0.10, with an IPC of 0.384632, while 0.20 and 0.25 remain very close.

These results indicate that the best threshold depends on the workload. However, threshold 0.20 provides performance close to the best observed IPC for all three traces, making it a reasonable shared configuration.

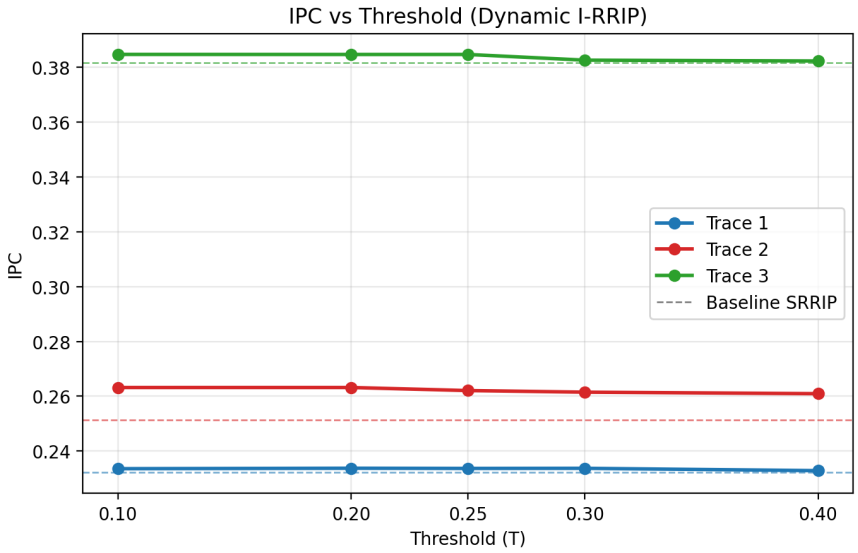



Figure 13: IPC versus threshold for Trace1, Trace2, and Trace3.

### 4.4 Speedup Analysis

Figure 14 shows speedup relative to the baseline IPC for each trace. All tested thresholds improve IPC over baseline in the measured results. The largest gain occurs for Trace2, with approximately 4.66% speedup at thresholds 0.10 and 0.20. Trace1 achieves a maximum speedup of approximately 0.67% at 0.20, while Trace3 achieves approximately 0.80% at 0.10.

The comparatively small improvements for Trace1 and Trace3 suggest that the effectiveness of instruction-conscious replacement depends on the workload's access behavior. Trace2 exhibits a stronger response to the

policy, indicating that its instruction and data access patterns benefit more from the adaptive replacement mechanism.

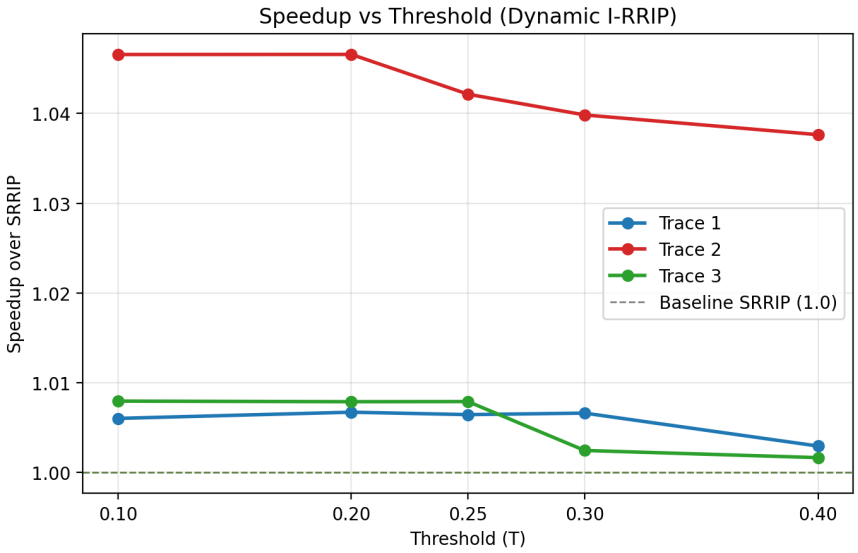



Figure 14: Speedup versus threshold for the three traces.

## 4.5 L2 Cache Load MPKI Analysis

Figure 15 compares total L2 load MPKI across the tested thresholds. For Trace1, total load MPKI increases relative to baseline at every tested threshold, despite the modest IPC improvement. In contrast, Trace2 exhibits a substantial reduction, from 26.4646 MPKI in the baseline to a minimum of 19.5767 at threshold 0.20. Trace3 also improves, decreasing from 4.0442 to approximately 3.87 for thresholds 0.10–0.25.

These results demonstrate that IPC and MPKI do not necessarily change in the same direction. Overall performance also depends on miss latency, the instruction/data access mix, and other effects in the simulated pipeline. In particular, a policy can improve IPC even when total load MPKI increases, as observed for Trace1.

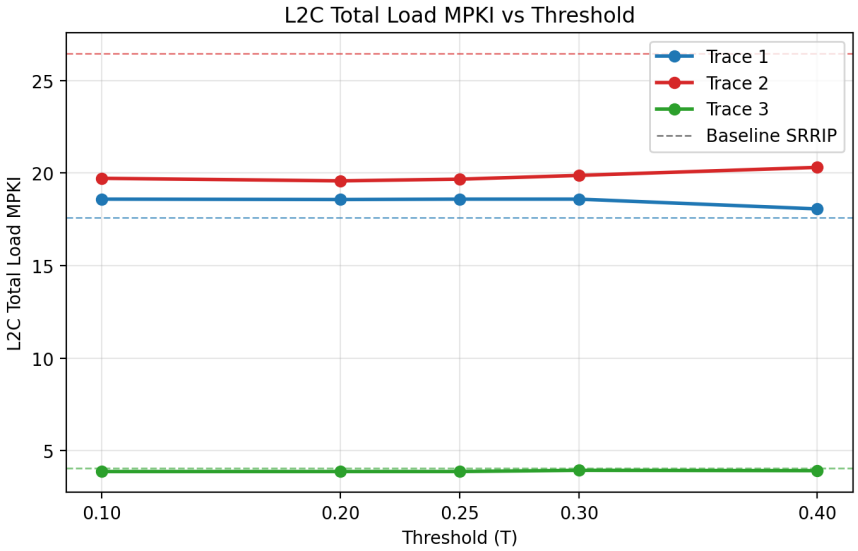



Figure 15: Total L2 cache load MPKI versus threshold for the three traces.

## 4.6 Instruction and Data MPKI Analysis

### 4.6.1 Instruction Load MPKI

Figure 16 shows instruction load MPKI. It decreases substantially from baseline for every trace across the tested threshold range. Trace1 decreases from 10.5392 to 8.9889 at threshold 0.20. Trace2 shows a larger reduction, from 18.3948 to 8.5286, while Trace3 decreases from 1.1996 to 0.7045.

These results are consistent with the design objective of protecting instruction lines in I-RRIP mode. The magnitude of the improvement depends on how frequently follower sets select I-RRIP under each threshold.

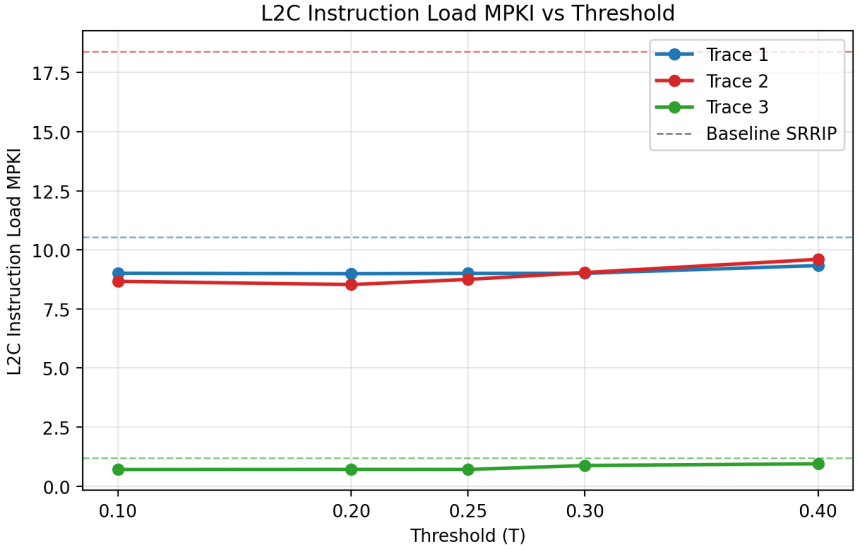



Figure 16: L2 instruction load MPKI versus threshold for the three traces.

### 4.6.2 Data Load MPKI

Figure 17 shows data load MPKI. Relative to baseline, data MPKI increases for all three traces at thresholds 0.10–0.30, while threshold 0.40 reduces this increase.

For Trace1, data MPKI increases from 7.06164 in the baseline to 9.58004 at threshold 0.20. For Trace2, it increases from 8.06980 to 11.04810, and Trace3 increases from 2.84460 to 3.16460. This behavior reflects the trade-off associated with providing greater protection to instruction lines: some data lines become more likely to be evicted.

Despite this penalty, the IPC results show that the trade-off can improve overall performance, particularly for Trace2. Therefore, instruction MPKI alone is insufficient to assess the effectiveness of the policy; data MPKI and IPC must also be considered.

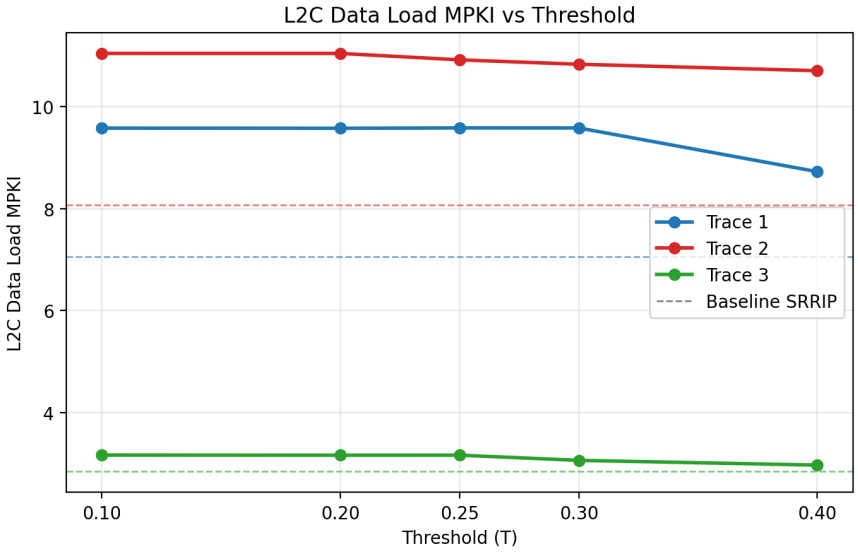



Figure 17: L2 data load MPKI versus threshold for the three traces.

## 4.7 Rationale for Leader-Set Selection and Decision Logic

The design selects one leader set for every 32 L2 cache sets. For a cache with 1024 sets, this produces 32 leader sets and 992 follower sets. This provides a small, regularly distributed observation group while leaving most of the cache available for adaptive replacement. Leader sets are not dynamically switched; they always use baseline RRIP, as required by the implemented set-dueling structure.

The decision metric follows the assignment's suggested form, using instruction accesses and misses relative to total accesses and misses. The threshold controls how readily follower sets enter I-RRIP mode. A lower threshold makes the condition easier to satisfy and can increase the use of instruction-conscious replacement, whereas a higher threshold makes the policy more conservative.

Counter halving after 8192 total leader-set accesses limits the persistence of older observations and allows more recent workload phases to influence policy selection. This is useful when the instruction/data access mix changes during execution.

The threshold sensitivity results support selecting 0.20 as the shared configuration. It produces the highest measured IPC for Trace1 and Trace2 and performs nearly as well as the best threshold for Trace3. Nevertheless, the differences among thresholds 0.10, 0.20, and 0.25 are small for several metrics. Therefore, the experiments support 0.20 as a reasonable configuration for the evaluated traces, but do not establish that it is optimal for every workload.

# 5 Task 3: Incorporating SPP with Dynamic I-RRIP

## 5.1 Objective and Motivation

Task 3 combines the Stride Pattern Prefetcher (SPP) implemented at the L1 data cache with the Dynamic Instruction-Conscious Re-Reference Interval Prediction (Dynamic I-RRIP) replacement policy at the L2 cache. The objective is to improve overall processor performance by considering not only whether a cache line contains instructions or data, but also whether the line was introduced speculatively by a prefetch request.

A prefetch can reduce memory-access latency when the predicted line is subsequently demanded. However, an inaccurate or late prefetch can occupy valuable cache capacity and evict a line that would otherwise have been reused. Treating every prefetched line like a frequently reused demand line may therefore increase cache pollution.

To address this problem, we incorporate prefetch awareness into `replacement/dyn_irrip.l2c_repl`. The replacement policy tracks lines brought in by prefetch requests and distinguishes them from lines that have demonstrated actual demand reuse. It preferentially evicts unused prefetched lines when they become sufficiently old, while retaining the existing Dynamic I-RRIP mechanism for ordinary demand accesses.

The implementation is evaluated on all three supplied traces. We compare the combined policy against a no-prefetching LRU baseline, Dynamic I-RRIP without SPP, and standalone SPP with LRU replacement.

## 5.2 Implementation and Design Rationale

### 5.2.1 Dynamic Instruction-Conscious Replacement

The existing Dynamic I-RRIP mechanism is retained for normal cache accesses. In I-RRIP mode, instruction lines receive more protection than data lines. A demand-filled instruction line is inserted with RRPV 2, whereas a demand-filled data line is inserted with RRPV 3. On a demand hit, instruction lines are promoted to RRPV 0, while data lines are promoted to RRPV 2. Writeback hits leave the replacement state unchanged.

Here, RRPV ranges from 0 to 3, where a higher value indicates that a line is predicted to be less likely to be reused in the near future. A line with RRPV 3 is eligible for eviction under the standard RRIP victim selection procedure.

The policy uses set dueling to decide whether follower sets should use I-RRIP or ordinary RRIP. Every 32nd L2 set is a leader set and always uses the baseline RRIP replacement policy. This corresponds to 32 leader sets when the L2 cache contains 1024 sets.

The policy tracks instruction accesses, instruction misses, total demand accesses, and total demand misses observed in the leader sets. Its decision metric is

$$R = \frac{N_{\text{instr,access}} + N_{\text{instr,miss}}}{N_{\text{total,access}} + N_{\text{total,miss}}}.$$

If $R > 0.20$, follower sets use I-RRIP; otherwise, they use standard RRIP. Writebacks and prefetch packets are excluded from these counters, so speculative traffic does not directly influence the instruction-versus- data decision. The counters are periodically halved after 8192 leader-set demand accesses, allowing the metric to adapt to changes in workload behaviour without allowing the counters to grow indefinitely.

### 5.2.2 Identifying Prefetched Cache Lines

The implementation maintains a Boolean array named `is_pref_block`, indexed by cache set and way. This array records whether a cache line was brought in by a prefetch request and has not yet been accessed by a demand request.

The packet type is checked using `type == PREFETCH`. This is important because a prefetched line should not automatically receive the same protection as a line whose usefulness has already been demonstrated by a demand access.

### 5.2.3 Insertion Policy for Prefetch Packets

When a prefetch packet fills an L2 cache line, the implementation sets `is_pref_block[set][way]` to `true` and initializes its RRPV to 2. The values of both `PREF_IRRIP_FILL_RRPV` and `PREF_RRIP_FILL_RRPV` are 2.

This choice is deliberate. Inserting a prefetched line immediately at RRPV 3 would make it an immediate eviction candidate, potentially discarding a correct prefetch before the processor has an opportunity to use it. Conversely, inserting it at RRPV 0 would give speculative data the strongest possible protection despite the absence of evidence that the prediction is correct.

RRPV 2 provides an intermediate level of protection. The line can remain in the cache long enough to be useful, but it does not receive the strong protection associated with a recently reused instruction line. The same initial RRPV is used in both replacement modes to keep prefetch insertion behaviour consistent.

#### 5.2.4 Promotion Policy for Prefetch Packets

The implementation handles prefetch hits separately from demand hits. When a prefetch packet hits an existing cache line, the replacement state is left unchanged, and the function returns immediately.

A prefetch hit alone is not sufficient evidence that the processor has consumed the prefetched line. Promoting the line on such a hit could unnecessarily protect speculative data and increase cache pollution. Therefore, the implementation does not allow a prefetch-only hit to improve the line's RRPV.

When a subsequent demand access hits a prefetched line, its `is_pref_block` flag is cleared. The normal replacement policy then determines its promotion. If the current mode is I-RRIP, the line is promoted according to whether it contains instructions or data. Otherwise, standard RRIP promotion is applied.

This distinction allows a prefetched line to earn protection through actual demand reuse rather than receiving that protection simply because it was fetched speculatively.

#### 5.2.5 Eviction Policy for Unused Prefetches

The victim-selection procedure introduces a separate priority for prefetched lines in follower sets. When a replacement is required, the policy proceeds as follows:

1. **Evict an aged, unused prefetched line first.** The policy searches for a line whose `is_pref_block` flag is set and whose RRPV equals 3. If such a line exists, it is selected as the victim.
2. **Apply instruction-conscious eviction.** If the follower set is operating in I-RRIP mode and no eligible unused prefetch has been selected, the policy searches for a non-instruction line with RRPV 3. This preserves the existing preference for retaining instruction lines.
3. **Fall back to ordinary RRIP.** If neither priority identifies a victim, the policy selects a line with RRPV 3 using the normal RRIP procedure. If no line has RRPV 3, the RRPVs are incremented until a victim becomes eligible.

The unused-prefetch priority is applied only to follower sets. Leader sets continue to use the baseline RRIP victim-selection procedure. This preserves their role in set dueling.

The distinction between unused and demand-used prefetches is important. A prefetched line that has already served a demand access is no longer marked as an unused prefetch and is not subject to the special eviction priority. The policy consequently targets speculative lines that have not demonstrated usefulness, instead of indiscriminately penalizing all lines introduced by the prefetcher.

### 5.3 Experimental Methodology

All simulations use 25 million warm-up instructions followed by 25 million simulation instructions, as specified in the assignment. The following four configurations are evaluated:

- **No-prefetch LRU:** no L1D data prefetcher and LRU replacement at L2C.
- **Dynamic I-RRIP:** the dynamic instruction-conscious replacement policy without SPP.
- **Standalone SPP:** SPP with LRU replacement at L2C.
- **SPP + Dynamic I-RRIP:** SPP combined with the prefetch-aware Dynamic I-RRIP implementation.

The IPC and cache statistics are obtained from the corresponding ChampSim simulation outputs. Speedup is calculated relative to the no-prefetch LRU baseline:

$$\text{Speedup} = \frac{\text{IPC}_{\text{configuration}}}{\text{IPC}_{\text{no-prefetch LRU}}}.$$

L1D load-miss coverage is calculated as

$$\text{Coverage} = \frac{M_{\text{baseline}} - M_{\text{configuration}}}{M_{\text{baseline}}} \times 100\%,$$

where $M$ is the number of L1D load misses in the simulation region. The baseline miss counts for Trace 1, Trace 2, and Trace 3 are 266,965, 338,529, and 85,093, respectively. Prefetch accuracy and the L1D, L2C, L2 instruction, and L2 data load MPKI values are taken from the reported simulator statistics. Accuracy and coverage are marked not applicable for configurations without a prefetcher.

**Configuration note:** The Dynamic I-RRIP-only results below are from the supplied `dirrip_exp3` runs, which use a threshold of 0.25 and 16 leader sets. The combined Task 3 runs use a threshold of 0.20 and 32 leader sets. Therefore, the comparison reflects the recorded configurations rather than a perfectly controlled experiment in which only prefetch awareness changes.

## 5.4 Experimental Results

### 5.4.1 IPC and Speedup

Table 20 presents the measured IPC and the speedup of each configuration relative to the no-prefetch LRU baseline.

Table 20: IPC and speedup for Task 3. Speedup is normalized to the no-prefetch LRU baseline.

| Trace | Metric | No-prefetch LRU | Dynamic I-RRIP | SPP + LRU | SPP + Dynamic I-RRIP |
|---|---|---|---|---|---|
| Trace 1 | IPC | 0.232264 | 0.233830 | 0.234229 | 0.235148 |
| | Speedup | 1.00000 | 1.00674 | 1.00846 | 1.01242 |
| Trace 2 | IPC | 0.250806 | 0.263344 | 0.251371 | 0.262809 |
| | Speedup | 1.00000 | 1.04999 | 1.00225 | 1.04786 |
| Trace 3 | IPC | 0.381942 | 0.384659 | 0.387850 | 0.389358 |
| | Speedup | 1.00000 | 1.00711 | 1.01547 | 1.01942 |
| Geometric mean | Speedup | 1.00000 | 1.02108 | 1.00871 | 1.02645 |

The combined policy achieves the highest IPC among the four configurations on Trace 1 and Trace 3. On Trace 2, its IPC is 0.262809, slightly below the Dynamic I-RRIP-only result of 0.263344, but substantially above the standalone SPP result of 0.251371.

Across all three traces, the combined policy achieves a geometric-mean speedup of 1.02645×, compared with 1.02108× for Dynamic I-RRIP and 1.00871× for standalone SPP. Thus, the combined configuration delivers the best aggregate performance among the evaluated configurations, although it does not outperform Dynamic I-RRIP on every individual trace.

### 5.4.2 Prefetch Accuracy, Coverage, and Load MPKI

Table 21 reports prefetch accuracy, L1D load-miss coverage, and the load MPKI at each requested cache level. The coverage values for SPP and the combined configuration use the same no-prefetch LRU baseline miss counts.

The combined policy reduces L1D load MPKI relative to the no-prefetch LRU baseline on all three traces. Its L1D load-miss coverage is 9.2480%, 1.8772%, and 15.9238% for Trace 1, Trace 2, and Trace 3, respectively. The corresponding prefetch accuracies are 53.4543%, 43.6320%, and 45.4745%.

The results also show that improvements at one cache level do not necessarily translate into a reduction in every other metric. For example, the combined policy has lower L2 instruction MPKI than standalone SPP on all three traces, while its L2 data MPKI is higher than standalone SPP on Trace 1, Trace 2, and Trace 3. The replacement policy therefore needs to balance instruction retention and demand-data reuse rather than minimizing only a single category of misses.

Table 21: Prefetch metrics and load MPKI for Task 3.

| Trace | Configuration | Accuracy (%) | Coverage (%) | L1D Load MPKI | L2C Load MPKI | L2 Instruction MPKI | L2 Data MPKI |
|---|---|---|---|---|---|---|---|
| Trace 1 | No-prefetch LRU | N/A | N/A | 10.67860 | 17.65390 | 10.33740 | 7.31648 |
| | Dynamic I-RRIP | N/A | N/A | 10.67990 | 18.59890 | 8.97152 | 9.62736 |
| | SPP + LRU | 53.9289 | 9.4173 | 9.67296 | 16.86300 | 10.37310 | 6.48992 |
| | SPP + Dynamic I-RRIP | 53.4543 | 9.2480 | 9.69104 | 17.48730 | 9.15364 | 8.33364 |
| Trace 2 | No-prefetch LRU | N/A | N/A | 13.54120 | 28.16620 | 19.50310 | 8.66312 |
| | Dynamic I-RRIP | N/A | N/A | 13.54900 | 19.67410 | 8.52560 | 11.14850 |
| | SPP + LRU | 45.4652 | 2.1035 | 13.25630 | 28.19890 | 19.79990 | 8.39904 |
| | SPP + Dynamic I-RRIP | 43.6320 | 1.8772 | 13.28700 | 19.54070 | 8.82256 | 10.71810 |
| Trace 3 | No-prefetch LRU | N/A | N/A | 3.40372 | 3.95580 | 1.12516 | 2.83064 |
| | Dynamic I-RRIP | N/A | N/A | 3.40380 | 3.86876 | 0.69824 | 3.17052 |
| | SPP + LRU | 44.4665 | 15.2363 | 2.88512 | 3.30492 | 1.14196 | 2.16296 |
| | SPP + Dynamic I-RRIP | 45.4745 | 15.9238 | 2.86172 | 3.19248 | 0.81668 | 2.37580 |

## 5.5 Plots

The following plots summarize the measured performance and cache behavior of the evaluated configurations.

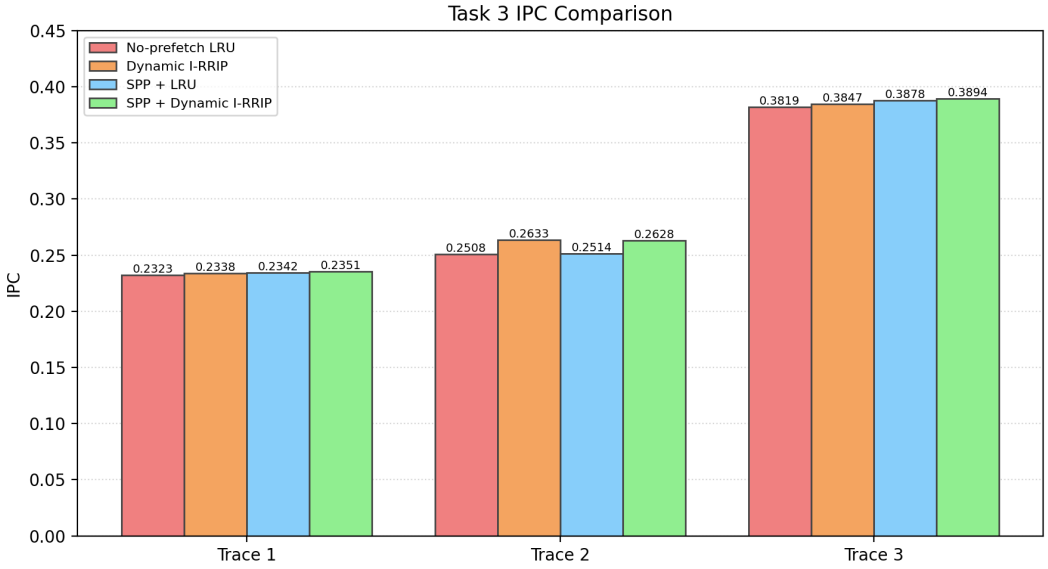



Figure 18: Task 3 IPC vs. Baseline

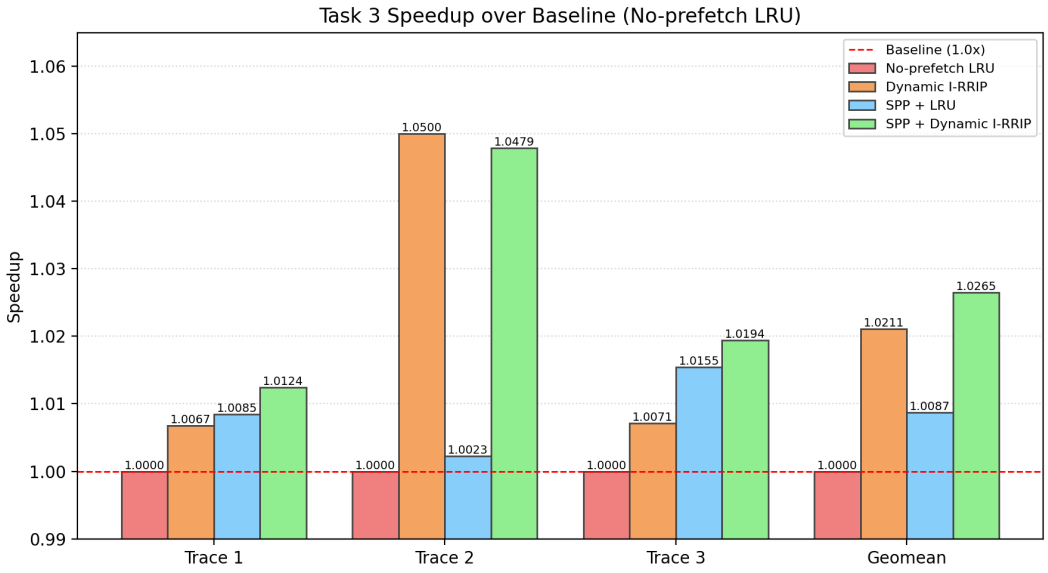



Figure 19: Speedup over Baseline

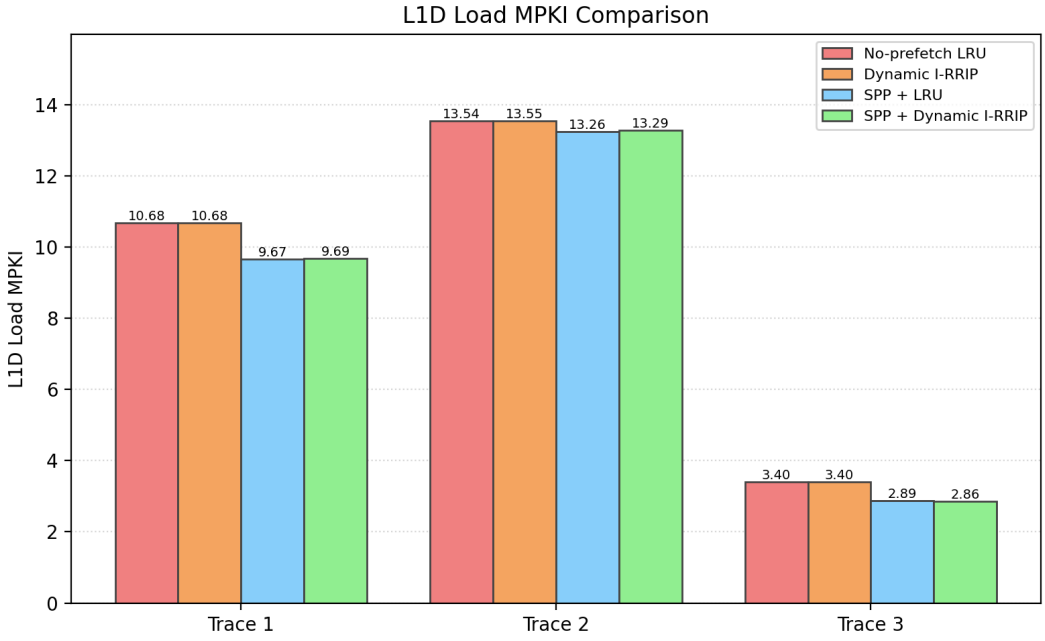



Figure 20: L1D Load MPKI Comparison

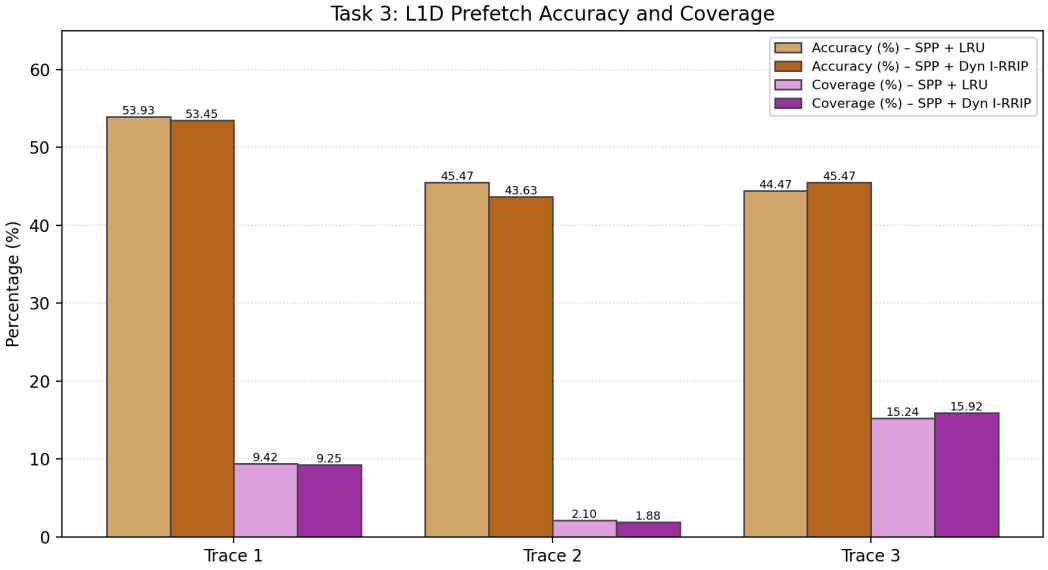



Figure 21: L1D Prefetch Accuracy and Coverage

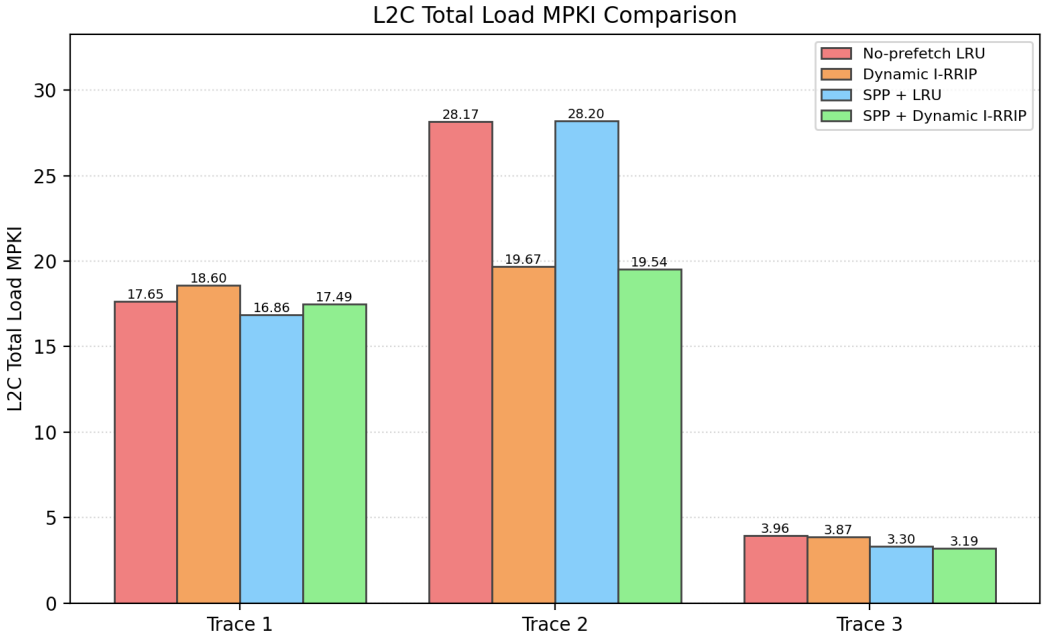



Figure 22: L2C Total Load MPKI Comparison

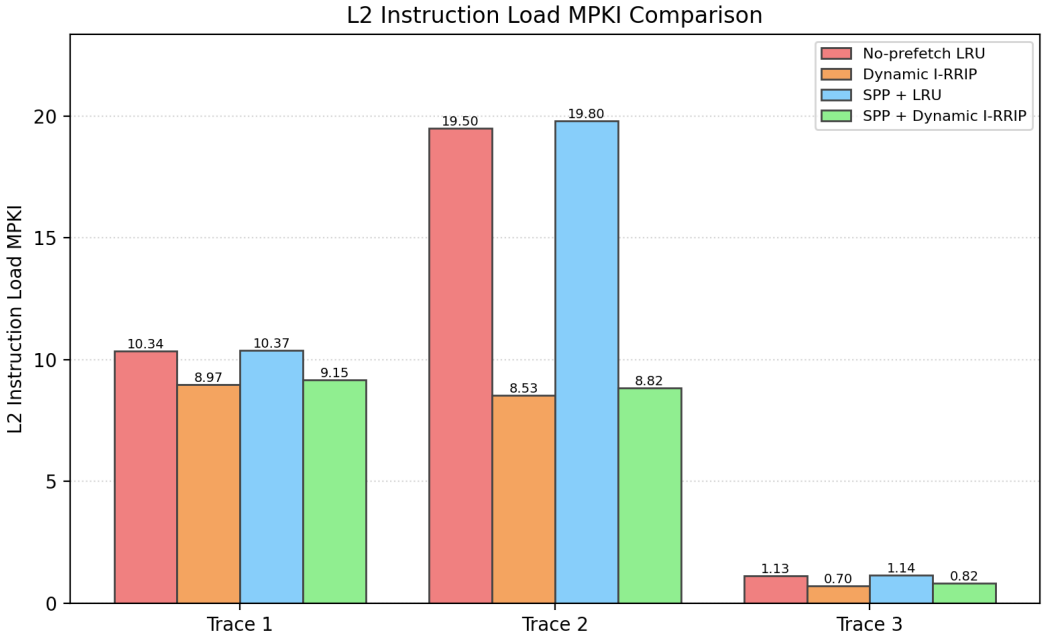



Figure 23: L2 Instruction Load MPKI Comparison

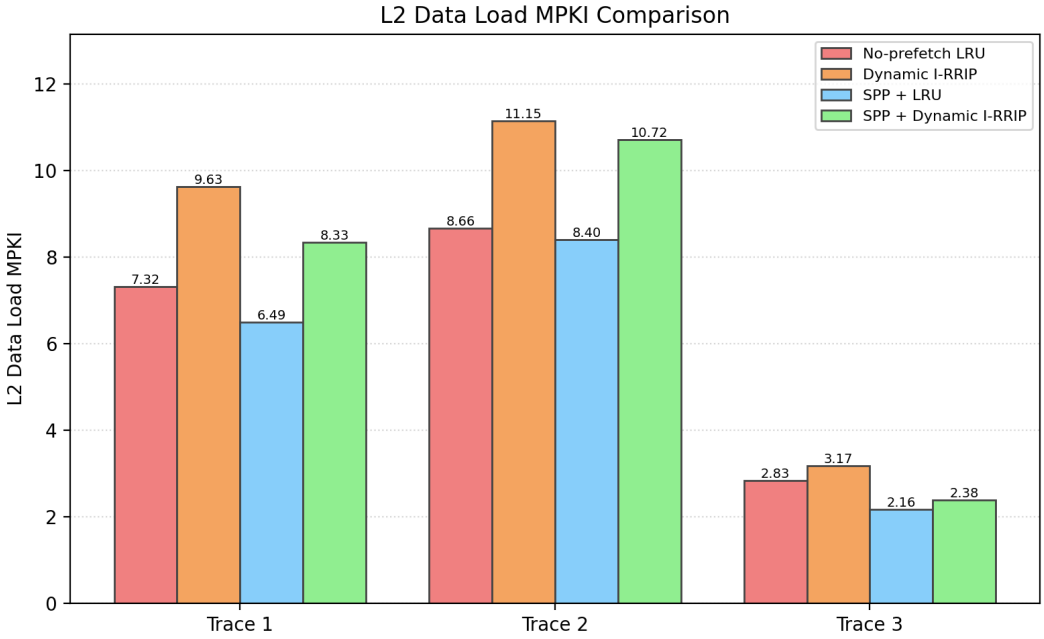



Figure 24: L2 Data Load MPKI Comparison

## 5.6 Discussion

The results demonstrate the benefit of combining prefetching with replacement-policy awareness. The combined implementation improves IPC over standalone SPP on all three traces and gives the best geometric-mean speedup among the evaluated configurations. Its reduction in L1D load MPKI also indicates that SPP continues to provide useful prefetches after the replacement policy is modified.

The eviction priority for aged, unused prefetched lines is intended to reduce the time for which speculative data occupies cache capacity after it has become unlikely to be useful. Not promoting prefetch-only hits further prevents speculative accesses from receiving unjustified protection. Once a demand access uses a prefetched line, however, the line is handled according to the normal dynamic replacement policy. This balances the opportunity to benefit from correct predictions against the need to preserve useful demand-fetched instructions and data.

The L2 results are workload-dependent. The combined policy reduces L2 instruction MPKI relative to standalone SPP on all three traces. However, the L2 data MPKI is higher than standalone SPP on these traces, showing the trade-off between instruction retention and data reuse. The combined policy should therefore be assessed using IPC and the full set of cache metrics rather than instruction MPKI alone.

Finally, although the combined configuration attains the highest geometric-mean speedup, it does not outperform Dynamic I-RRIP alone on Trace 2. In addition to workload-dependent behaviour, the standalone Dynamic I-RRIP and combined runs use different set-dueling parameters, as described in the experimental methodology. A stricter comparison of prefetch-awareness alone would require rerunning Dynamic I-RRIP with the same threshold and leader-set configuration as the combined policy.

## 5.7 Task 3 Summary

Task 3 extends Dynamic I-RRIP by distinguishing prefetched lines from demand-used lines. Prefetched lines are inserted at RRPV 2, prefetch-only hits do not promote replacement state, and aged unused prefetches receive first eviction priority in follower sets. Demand accesses clear the unused-prefetch marker and return the line to the normal RRIP or I-RRIP handling path.

Across the three evaluated traces, SPP + Dynamic I-RRIP achieves a geometric-mean speedup of 1.02645× relative to the no-prefetch LRU baseline. It outperforms standalone SPP in IPC for every trace and Dynamic I-RRIP on two of the three traces. These results support the usefulness of prefetch-aware replacement at an aggregate level while also highlighting the importance of workload behaviour and consistent experimental parameters when assessing individual-trace performance.